

UNIVERSITY OF ZIELONA GÓRA

Faculty of Computer, Electrical and Control Engineering

Bachelor's Thesis

Degree Course: Computer Science

**METHODS OF IMPLEMENTING A NETWORK
INTERFACE IN IoT MODULES**

Kacper Wojciechowski

Dissertation advisor:
PhD eng. Mirosław Koziół

Accepted by:

.....

(date and dissertation advisor signature)

Zielona Góra, January 2022

Summary

Following dissertation aims to provide an overview of available methods of implementing a network interface for IoT modules that utilize Cortex-M series microcontrollers. Presented were the Internet of Things field, substantiation of the use of network interface by said modules, structure and protocols contained within the TCP/IP stack, and commercially available solutions for implementing a network interface, such as Ethernet and 2.4 GHz band Wi-Fi modules. Additionally, one of the chapters provides an example implementation of Ethernet and 2.4 GHz band Wi-Fi connectivity using Nucleo platform by ST Microelectronics.

Keywords: mikrocontroller, ST Microelectronics, network interface, Ethernet, wireless interface.

Table of Contents

1. Prelude	1
1.1. Introduction	1
1.2. Aim and scope	1
1.3. Document structure	2
2. Internet of Things	3
2.1. Formal definition	3
2.2. Fields related to constructing and operating IoT modules	3
2.3. Advantages and disadvantages	4
2.4. IoT modules as the Internet network devices	5
3. TCP/IP stack	7
3.1. TCP/IP stack characteristics	7
3.2. Application layer	7
3.3. Transport layer	8
3.4. Network layer	9
3.5. Network access layer	10
3.5.1. Wired connectivity	11
3.5.2. Wireless connectivity	13
3.6. TCP/IP stack implementation	15
4. Practical implementation of the network interface	19
4.1. Project goals	19
4.2. Chosen components	19
4.2.1. Hardware platform	19
4.2.2. Environment and programming language	19
4.2.3. Verification methods of the system functionality	20
4.3. Hardware implementation	20
4.3.1. Ethernet interface	20
4.3.2. Wireless interface	20
4.4. Software implementation	21
4.4.1. Ethernet interface	21
4.4.2. Wi-Fi wireless interface	39
4.5. Solution tests	46
4.5.1. Ethernet interface	46
4.5.2. Wi-Fi wireless interface	47
5. Results discussion and conclusions	51

List of Figures

2.1. Graphical comparison of the ISO OSI and TCP/IP models	6
3.1. TCP transmission diagram	8
3.2. UDP transmission diagram	9
3.3. Packet path diagram in the OSI model - Computer Notes	10
3.4. 8P8C port line enumeration - Electronics Stack Exchange	11
3.5. 8P8C connector line enumeration - Electronics Stack Exchange	11
3.6. Functional diagram of DB83848-P Ethernet driver by Texas Instru- ments - Texas Instruments	12
3.7. DB83848-P driver placement - Texas Instruments	12
3.8. Line placement in UTP cable - The Internet Centre Inc.	13
3.9. Antenna types for embedded systems - Circuit Digest	14
3.10. Wi-Fi Shield - Seeed Studio	14
3.11. GSM Shield - Botland	14
3.12. ENC28J60 module - Aliexpress	15
3.13. W5500 module - Aliexpress	15
3.14. ESP8266 module - Nettigo	16
3.15. ESP32 module - Botland	16
3.16. Nucleo-F429ZI platform with integrated Ethernet port - Botland . . .	17
4.1. 8P8C connector present on the Nucleo-F429ZI board	20
4.2. ESP8266 connection schematic	21
4.3. Microcontroller selection in STM32CubeIDE	21
4.4. Creating the project	22
4.5. Switching the HSE clock into BYPASS mode	22
4.6. Configuring clocking signal after switching HSE to BYPASS mode . .	22
4.7. Assigning RMII mode to the Ethernet interface	23
4.8. Target values of RMII parameters	23
4.9. RMII interface connection schematics	24
4.10. Enabling LwIP stack	25
4.11. LWIP_NETIF_LINK_CALLBACK option	25
4.12. Clock configuration	25
4.13. USART3 and USART6 configuration	39
4.14. HSE clock setup	39
4.15. Target frequencies in the clock manager	40
4.16. Configured GPIO lines	40
4.17. PB3 line parameters	40
4.18. Serial port communication	47
4.19. Verifying connection via ping	47
4.20. Two-way communication validation using Hercules SETUP Utility . .	48

4.21. Constructed circuit	48
4.22. Communication with overseeing computer pt. 1	49
4.23. Communication with overseeing computer pt. 2	49
4.24. Testing connectivity using <i>ping</i>	49
4.25. Two-sided communication using Hercules SETUP Utility software . .	50

List of Tables

3.1. RMII lines	13
3.2. Ethernet module parameters comparison	16
3.3. Overview of wireless modules parameters	17
4.1. Output line assignment	24

Chapter 1

Prelude

1.1. Introduction

Along with the progress in technology, the market is populated by more and more innovative inventions that utilize previous achievements of information technology and that drive the technological progress forward. IT industry is one very important example of this phenomenon. Over the years, increasingly clearly can be observed the presence of electronic and computing devices in many parts of the daily life, starting with education, through medical care, ending on entertainment. The development of the Internet plays a major role here, allowing to connect many devices into extensive networks for information exchange.

Many of currently commercially available products are a derivative of ever increasing automatization and widening benefits of media such as the Internet. One field that presents this very well is the Internet of Things (*IoT*), consisting of many distributed hardware modules connected to a common computer network in order to create a larger system.

1.2. Aim and scope

The aim of this dissertation is to provide an overview of network interface implementation methods available for IoT modules and showing practical implementation (both in hardware and software) of selected methods in a system based on Cortex-M microcontroller.

The scope of the thesis consists of:

- overview of possible methods of implementing a network interface in IoT modules based on microcontrollers with Cortex-M processing unit, including pros and cons of each of them;
- practical implementation of IoT module based on microcontroller with Cortex-M processing unit and network interface realized according to one of the selected methods.

1.3. Document structure

The first chapter contains an introduction to the topic, aim and scope in which this thesis was made, and short information about each section of this document.

The aim of the second chapter is to familiarize the reader with the term Internet of Things (IoT) along with its formal definition, presenting the pros and cons of this field. This section was completed with substantiation of implementing network interfaces in IoT modules as network devices.

The third chapter discusses the TCP/IP stack and its relation to the ISO OSI model. The content presents and shortly describes the protocols available in each of the layers of TCP/IP model. Additionally, a difference between TCP and UDP transmission is presented, and physical layer connection method is discussed.

The topic of the fourth chapter is a practical implementation of Ethernet and 2.4 GHz band Wi-Fi connectivity for Nucleo-F429ZI hardware platform. This chapter provides step by step explanation of the device configuration process and code procured for handling the interface.

The last chapter discusses the results of this thesis and presents the conclusions drawn during its realization.

Chapter 2

Internet of Things

2.1. Formal definition

According to the definition provided in the ITU-T Y.2060 recommendation, the Internet of Things was described as "global infrastructure for society, allowing the operation of advanced services through the connection (physical and virtual) of subjects based on existing and evolving interoperative communication and information exchange technologies" [1].

In practical approach, the above definition pictures the Internet of Things as complex systems providing advanced services through communication and cooperation of various devices using different transmission media, such as the Internet.

2.2. Fields related to constructing and operating IoT modules

Internet of Things modules are the product of cooperation of specialists from many fields. While omitting the branches of science and industry directly connected to given functionality of a specific system, we can identify the following fields, the presence of which can be observed when analyzing various systems:

- **Embedded systems** – they are lying at the very base of the functionalities offered by IoT modules. Each device creating the discussed networks must be equipped with digital circuitry allowing it to perform the tasks put before them, such as taking measurements or monitoring the operating parameters of another device. This purpose is realized using microcontrollers, which control peripheral devices (such as sensors, displays, etc.), perform preprocessing of gathered data, and allow for communication with the device using various interfaces (including serial interfaces such as SPI, I²C, and network interfaces).

- **Data analysis** – one of the primary tasks of IoT modules is gathering and generating data, which subsequently needs to be processed by units with more computing power. Depending on the size of the data and time requirements regarding the processing speed, this step can be implemented at different layers in the system hierarchy. In order to filter, organize and infer based on the data, algorithms and mathematical formulas are widely used, allowing eg. to combine related information and present them in a way that allows for drawing more in-depth conclusions.
- **Artificial intelligence** – the creation based on mathematical rules for data analysis which aim to drastically speed up and automate the processing of data, by mimicking biological intelligence. It allows for interpreting tremendous amount of data, in the time frame which substantially exceeds the capability of the entire teams of human analysts. This translates to significant improvement of the IoT systems efficiency. Additionally, artificial intelligence often is one of the integral functionalities of the system provided to the user, especially in intelligent systems such as *smart home* or security systems (eg. by allowing to identify people based on video or biometric data).
- **Computer networks** – due to the need for exchanging data between devices, which is lying and the core of the Internet of Things concept, each module needs to be equipped with one or more communication interfaces in order to allow for data transfer both from and to the device. Examples of such interfaces are wired Ethernet and wireless Wi-Fi and GSM connectivity. Those interfaces require additional security measures via encryption.
- **3D printing** – IoT modules often perform innovative and non-standard tasks, which limits the possibility of reusing the elements commonly available on the market, such as chassises, fastening elements and parts of moving mechanisms. Here, the 3d printing industry provides a dramatic advantage, allowing for fast and easy procurement of both prototype elements, and parts for the final products, significantly reducing the production costs.

2.3. Advantages and disadvantages

The Internet of Things systems are innovative and very flexible invention, and as such, they are characterized by many advantages, among which the major ones are:

- being the space for innovative ideas impacting society living standards, and the efficiency of industry and scientific research;
- increasing the safety of facilities and personel, which is especially important in work places with higher risk factors;
- allowing the monitoring of changes taking place in the surroundings according to various factors and time;
- gathering valuable measurement data, which are too extensive and extremely hard to detect directly by a human (eg. determining a specific level of air pollution).

A large number of presented advantages emerge from the way of constructing the modules, and designing the Internet of Things systems, utilizing various elements with wide range of applications for constructing circuits for specific purposes. One such example could be the *smart home* systems, which thanks to the combination of elements such as humidity sensor, smoke sensor and vibration sensor are capable of overseeing the security of the building (protecting both from a fire, as from an intrusion).

Similarly to other technologies, IoT systems are not free from faults which result from the current technological advancement, component quality or design flaws of created solutions. The most important of them are:

- drastically limited computing power of the end measurement usings, requiring a connection with a central hub with significantly higher compiting capabilities in order to process big data packets;
- a necessity to buy a whole set of devices in order to make a complete, fully functional system, which detracts individual clients;
- the need to handle communication with several devices separated by various distances (sometimes very large distances, especially in case of industrial and environmental systems).

The ratio of pros to cons of the Internet of Things solution is a subject of many discussions and entirely depends on the specific use cases.

2.4. IoT modules as the Internet network devices

The Internet is a very popular transmission medium in many homes, offices and industrial facilities. Its concept – which is one of the core ideas behind the Internet of Things – caused that the majority of current IoT modules utilizes the Internet access as a cornerstone of the system it is a part of. It can be observed especially in case of the systems of individual clients and office spaces.

In order to use the communication via Internet network IoT modules have the need to implement the communication stack which is based on the OSI model (*Open System Interconnection Reference Model*) issued by the ISO (*International Organization for Standarization*) commission. It consists of a 7-layer hierarchy of a communication system elements, divided based on their roles. According to the presentation made by the employee of the Cathedra of Intelligent Information Systems of Politechnika Częstochowska, associate professor Robert Nowicki [2], each layer of the OSI model (starting from the top) execute the following functions:

- 1) application layer - handles the typical functionalities of the program, such as browsing the WWW websites and providing common API for various services;
- 2) presentation layer - formats data to a common representation using predefined syntax;
- 3) session layer - notifies the user of errors which may occure at the lower layers, enforces syntonization and manages sessions between services;

- 4) transport layer - oversees the connections present in the network layer, and their associated events, such as timeout and packet sequence validity;
- 5) network layer - provides addressing functionality, divides and reassembles packets from fragments, and decides on the route through which a packet will be sent;
- 6) data link layer - creates the transmission frame, establishes the rules of transmission and oversees the data exchange in physical layer;
- 7) physical layer - makes up the physical medium for transmitting the data.

Based on the described model there has been created the TCP/IP communication protocol stack, in which the application, presentation and session layer were collectively called the application layer, and the data link and physical layers were combined into a network interface layer, creating a 4-step hierarchy. Each layer contains a group of protocols responsible for adapting the transmitted data through *encapsulation*, and transforming received data from the form of a transmission frame into the representation which is useful for the end service (eg. a web browser) [3]. Graphical comparison of layers of both models was shown on the figure 2.1.

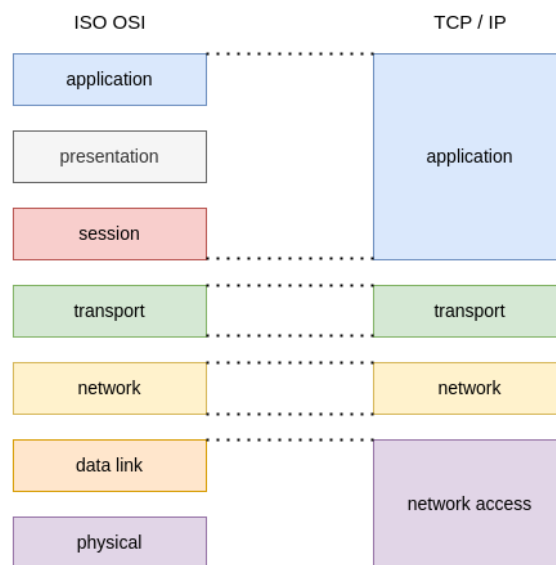


Figure 2.1. Graphical comparison of the ISO OSI and TCP/IP models

Chapter 3

TCP/IP stack

3.1. TCP/IP stack characteristics

As a simplified OSI model, the TCP/IP stack allows for simple classification of the protocols based on their functions and abstraction level. Lower layers focus primarily on the hardware aspects, while higher levels target functionalities. This chapter discusses each layer of the TCP/IP stack in a descending order.

3.2. Application layer

This layer serves as an interface between computer software and the remaining TCP/IP stack layers. Depending on the functionality provided by the end program, application layer puts forward protocols of various uses [4]:

- 1) HTTP, HTTPS (*Hypertext Transfer Protocol, Hypertext Transfer Protocol Secure*) – protocols responsible for transmission of WWW forms between the server and the end user, in non-encrypted and encrypted version respectively [5];
- 2) FTP, TFTP, NFS (*File Transfer Protocol, Trivial File Transfer Protocol, Network File System*) – protocols for transmitting files between two devices via the Internet;
- 3) SMTP, POP3, IMAP (*Simple Mail Transfer Protocol, Post Office Protocol 3, Internet Message Access Protocol*) – a group of protocols responsible for e-mail communication [6][7];
- 4) Telnet – a protocol for remote access into different computer, allowing for logging in and transmitting input from host to the remote client [8];
- 5) SNMP (*Simple Network Management Protocol*) – protocol for managing devices connected to the network, which are equipped with SNMP agent. It allows for the remote configuration of network devices such as routers, servers and switches [9];
- 6) DNS (*Domain Name System*) - a protocol responsible for translating the domain names into IP addresses of the servers hosting given website or service;

- 7) IRC (*Internet Relay Chat*) – a client-server model protocol which allows live communication via text communicators (*chats*) [10];
- 8) DGCP (*Dynamic Host Configuration Protocol*) – it is responsible for dynamically configuring the end devices connected to a computer network through assigning them so called dynamic IP address based on the available address pool.

Regardless of the different form of information created by the aforementioned protocols, lower layers interpret it as an universal data section, which is subsequently wrapped by lower layer protocols in the encapsulation process. In order to communicate between the application and transport layer, so called *Internet Sockets* are used, which are software constructs containing information such as IP address, port number and used protocol [4].

3.3. Transport layer

Here data received from the application layer is divided into a series of packets and wrapped in a header containing information such as the number or the port of the sender and receiver, which is important in order to determine, which protocol of the application layer should get the message on the receiver end. Protocols used in this layer determine the characteristics of the transmission. The main protocol of this layer are TCP and UDP.

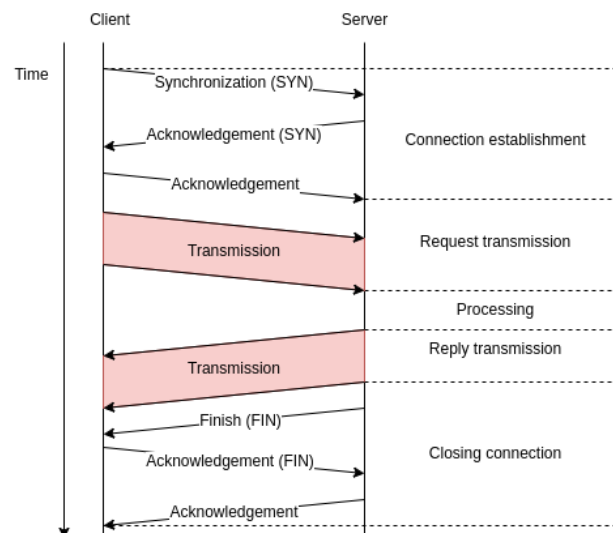


Figure 3.1. TCP transmission diagram

TCP (*Transmission Control Protocol*) is a connection oriented protocol, which means that at the start of the transmission, a session between the transmitter and the receiver is created. Additionally, it is characterized by reliability, due to the requirement of getting an acknowledgement for each transmitted packet, and performing a retransmission in case of transmission errors or lack of acknowledgement.

This characteristics is especially important in case of exchanging data, which integrity plays a key role, such as files or e-mail messages. Primary protocols in the application layer which utilize TCP transmission are: HTTP, HTTPS, FTP, SMTP and Telnet [4]. The flow of a singular transmission is presented on figure 3.1.

UDP (*User Datagram Protocol*), on contrary to the TCP, is not a connection oriented protocol and it does not perform any retransmissions. This means that transmitting a message using the UDP protocol does not guarantee that it reach the recipient. Additionally, the order in which the packets are received does not need to reflect the order of their transmission, which requires sorting by the recipient devices. The advantage of UDP protocol however is increased speed of the transmission thanks to significant simplification of utilized mechanisms. This comes especially handy in case of services, where losing singular packets is not a problem, and keeping a low latency is of much bigger importance. This especially concerns voice transmission, and online video games. Example protocols utilizing UDP transmission are: DNS, DHCP, TFTP, SNMP, RIP and VOIP [4]. The flow of singular transmission was presented on figure 3.2.

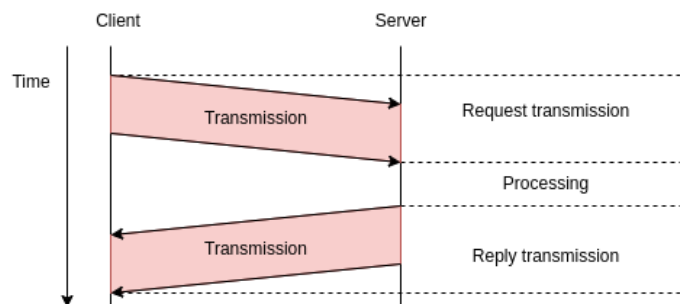


Figure 3.2. UDP transmission diagram

3.4. Network layer

In this layer, the transmission frame is wrapped in address fields containing the IP addresses of the sender and the recipient. Their presence is important due to the method of routing data in the Internet, which bases on passing the packet between subsequent devices until it reaches the recipient. Network devices know on which port the packet should be forwarded thanks to their configuration. The IP protocol used for addressing is available in version IPv4 and IPv6, which differ in the length of the addresses, the manner of addressing networks and subnetworks, and translation of the addresses between public and private in case of IPv4, or lack thereof in case of IPv6.

Other than the IP protocol, the network layer offers following protocols:

- 1) ARP (*Address Resolution Protocol*) – it serves the purpose of acquiring the MAC address of the device with corresponding IP address;
- 2) RARP (*Reverse Address Resolution Protocol*) – allows for acquiring the IP address of a device based on its MAC address;

- 3) ICMP (*Internet Control Message Protocol*) – a protocol allowing to oversee the state of given computer network.

The necessity of use of protocols which give access to MAC addresses based on IP addresses arises from the difference in addressing between the network layer, which uses IP addresses, and network access layer, which uses MAC addresses. During the transmission of a packet, each of the intermediary devices performs packet decapsulation to the form available in network layer, and subsequently makes the decision whether the packet should be forwarded further based on the IP address. In case of the need to forward the message to another device, it is encapsulated again with information about MAC addresses in the network access layer. In the opposite case the packet is forwarded to the higher layers. Figure 3.3. shows the layer diagram, in which a packet is processed during transmission between two intermediary devices.

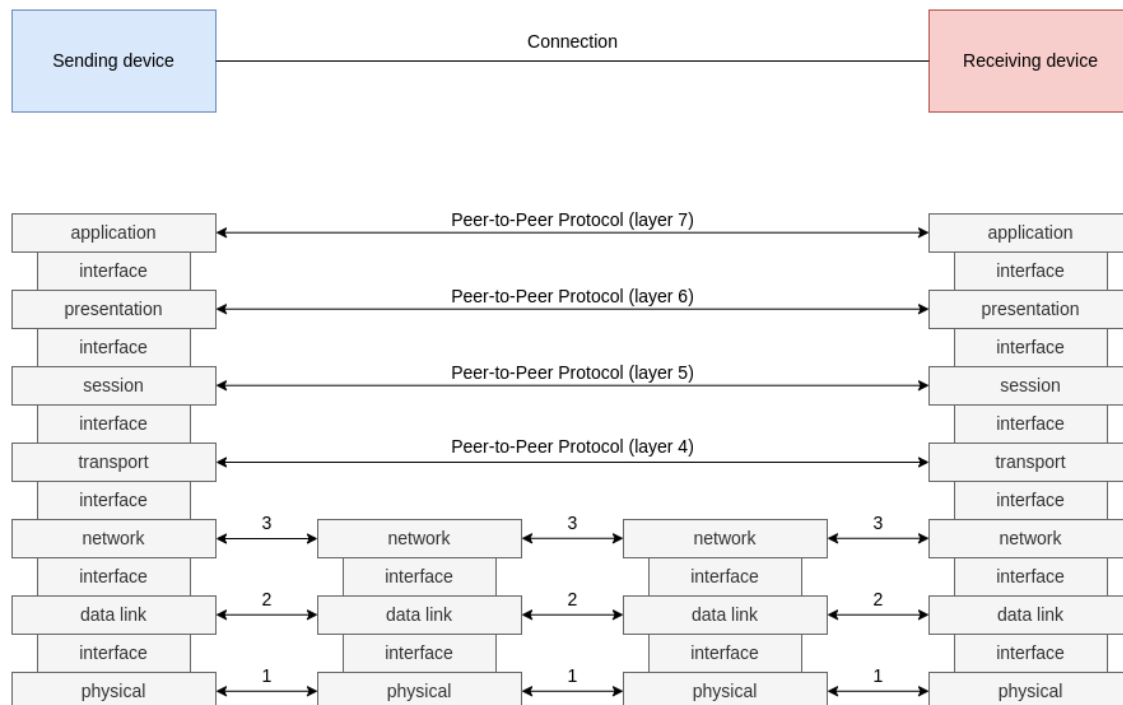


Figure 3.3. Packet path diagram in the OSI model - Computer Notes

3.5. Network access layer

The layer responsible for preparing the packets for transmission using the physical medium, and carrying out the transmission. Both in case of wired and wireless networks, following sublayers can be identified:

- 1) LLC (*Logic Link Control*) – concatenates the frame with information about the protocol used in transport layer, in order to allow the recipient device to correctly interpret the received packet;

- 2) MAC (*Media Access Control*) – creates the physical addresses sections in the Ethernet frame, storing MAC address of the sender and either the recipient, or intermediary router. In case of intermediary routers, the recipient address is updated by the router during the forwarding of the packet;
- 3) PHY (*Physical*) – consists of elements of the physical medium for transmission, such as transmission wires, antennas and transceivers;

Two primary connectivity methods can be distinguished on the physical interface level: wired and wireless/

3.5.1. Wired connectivity

In order to connect devices in a wired manner, an UTP cable is commonly used, which utilizes the 8P8C connector, commonly incorrectly called RJ-45 due to very similar shape (RJ-45 connector has additional extrusion) [12]. The 8P8C is an 8-pin connector, which has 4 pairs of wires marked by straight and dashed isolation colors. It allows for connecting two devices via Point-to-Point method, using the PPPoE protocol (*Point-to-Point Protocol over Ethernet*). This connector is often found both in form of extension modules which use serial interfaces, as in form of integrated port on the circuit board. Figure 3.4 presents the enumeration of 8P8C port lines, and figure 3.5 presents the enumeration for 8P8C connector.



Figure 3.4. 8P8C port line enumeration - Electronics Stack Exchange

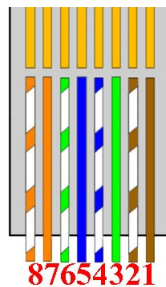


Figure 3.5. 8P8C connector line enumeration - Electronics Stack Exchange

The connection between the MAC and the PHY layers is carried out via platform-independent MII interface (*Media-Independent Interface*) or its reduced version RMII (*Reduced Media-Independent Interface*). It takes form of a connection between the processing unit and the Ethernet port driver, equipped with DAC (*Digital-Analog Converter*) and ADC (*Analog-Digital Converter*) in order to allow for transmission via differential signal through the Ethernet cable. Figure 3.6 presents a functional

diagram of a sample Ethernet driver, and figure 3.7 its placement in a platform blueprint between MAC and PHY layers.

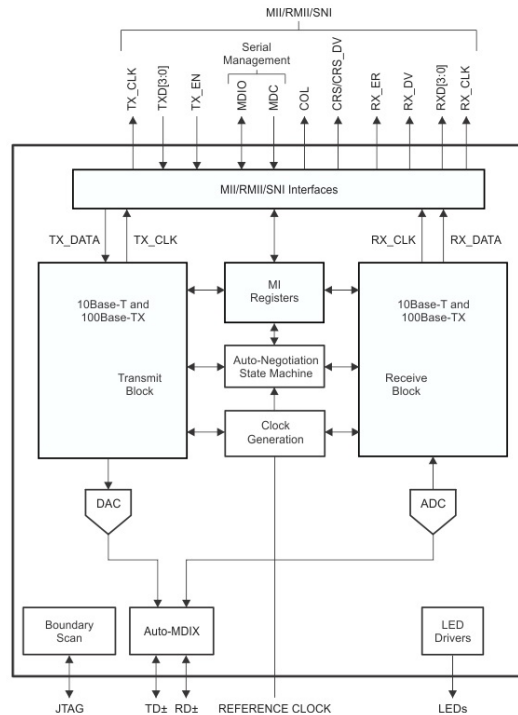


Figure 3.6. Functional diagram of DB83848-P Ethernet driver by Texas Instruments - Texas Instruments

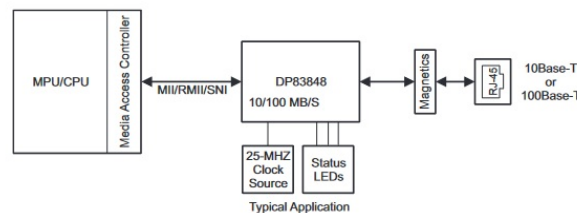


Figure 3.7. DB83848-P driver placement - Texas Instruments

The use of RMII allows for connecting different physical mediums (such as UTP cable, fibre optics) to the same data link layer. Table 3.1 describes the lines present in the RMII and their purposes [13][14]. The MAC abbreviation stands for data link layer, and the PHY abbreviation stands for physical layer.

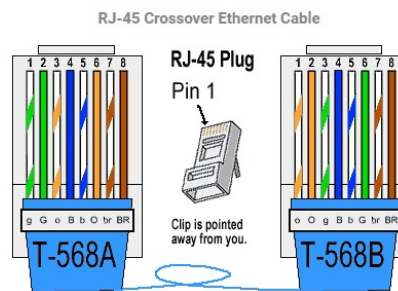
Transmitting via RMII requires converting the binary representation of the code of sent character to big-endian notation, which even index bits (starting enumerating from 0) are transmitted via line TXD0, and odd index bits via line TXD1, creating so called *di-bit* or *nibble* transmission. Similarly to the transmission, reception of the data happens by receiving bits with even index (with enumerating from 0) on line RXD0 and bits with odd indexes on line RXD1.

The connection of lines within the UTP cables for the 8P8C connectors is an important matter. In case of connecting two devices deemed as *hosts*, it is necessary to use a crossover cable, which is a cable where the transmitting and receiving pair between each end of the cable are switched. However in case of connecting a host

Table 3.1. RMII lines

Line designation	Transmission direction	Description
REF_CLK	Any one-way	Reference clock 50 MHz
TXD0	MAC -> PHY	First transmission bit
TXD1	MAC -> PHY	Second transmission bit
TX_EN	MAC -> PHY	Transmission start signal
RXD0	PHY -> MAC	First reception bit
RXD1	PHY -> MAC	Second reception bit
CRS_DV	PHY -> MAC	Carrier detection line
RX_ER	PHY -> MAC	Reception error line
MDIO	two-way	Administrative data line
MDC	MAC -> PHY	Clock signal for line MDIO

to a device such as a router (which has an on-board switch), a straight-thru cable needs to be used. Those two ways of making the 8P8C connector for the UTP cable are known as T-568A and T-568B respectively.

**Figure 3.8.** Line placement in UTP cable - The Internet Centre Inc.

3.5.2. Wireless connectivity

Thanks to the popularity of integration of embedded systems with wireless networks, especially in case of home and office automation, the market offers a number of different modules providing wireless network functionalities, both for Wi-Fi and cellular data. Due to their very similar construction, in which the major difference often is only the type of antenna for given frequency, this thesis provides only a general description of said modules.

Majority of the modules are available in the form of preprogrammed digital circuits on a PCB board, most commonly with integrated microband antenna (in form of a path on the printed board), with SMA antenna port used for concentric cables, IPEX port, or u.FL port. Figure 3.7 presents example antennas used by wireless modules.

Part of the circuits which implement the wireless interface are available in a form of a Shield extension, which allows for very easy integration with Arduino hardware platform, without the need for soldering. The construction of such module is very similar to the Arduino hardware platform itself, having ports that are corresponding with function and placement to the ones present on the microcontroller board. In

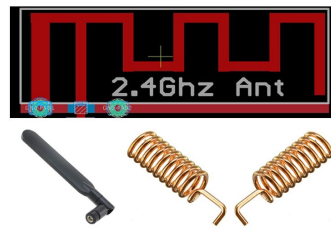


Figure 3.9. Antenna types for embedded systems - Circuit Digest

case of modules which offer the GSM connectivity, it is necessary to additionally provide a SIM card, similarly to the common mobile phones.

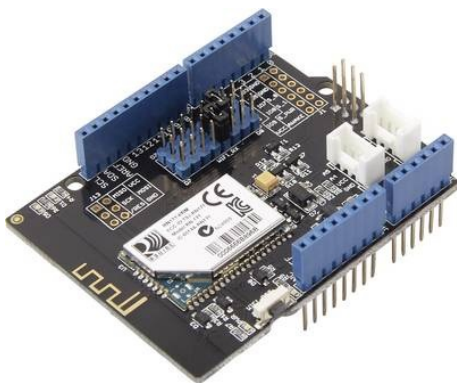


Figure 3.10. Wi-Fi Shield - Seeed Studio



Figure 3.11. GSM Shield - Botland

3.6. TCP/IP stack implementation

Implementation of TCP/IP stack, both due to the number and the complexity of the protocols, is a complex task. Because of that, the most common approach is to use already established solutions. Some of the devices (especially extension modules) implement the whole TCP/IP stack, requiring from the user only implementing the communication with selected local serial interface, such as SPI or I²C.



Figure 3.12. ENC28J60 module - Aliexpress

ENC28J60 module [15] is an example of solution providing an external Ethernet interface, which was shown on figure 3.12. It has compatibility with 10/100/1000Base-T networks, supporting especially 10Base-T transmission. In order to communicate with a microcontroller, and SPI interface with the clock frequency of 20 MHz is used. It provides only the implementation of physical and data link layer of the OSI model, which forces the user to implement the other layers of the TCP/IP stack on their own. Said module possesses an 8kB programmable memory for reception and transmission buffers and hardware control sum calculation. Thanks to implementing the application layer on the microcontroller, the amount of available network logical sockets is entirely dependent on the RAM memory size of the microcontroller.

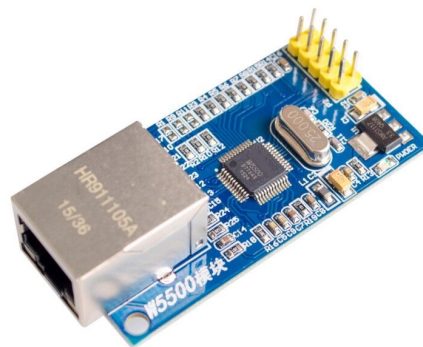


Figure 3.13. W5500 module - Aliexpress

W5500 is another example of wired connectivity as shown on figure 3.13. On contrary to the ENC28J60 module, it offers also part of the higher layers protocols, such as TCP, UDP, ICMP, IPv4, ARP, IGMP and PPPoE [16]. Additionally it provides the functionality of *wake on LAN*. This module is also available in the

form of Shield extension for the Arduino boards. It operates with standards 10/100 Base-T and contains 32 kB programmable memory for transmission and reception buffers, and supports 8 logical network sockets. The module operates with 3.3 V power level, while tolerating 5 V on the I/O (Input/Output) pins. Table 3.2 presents the comparison of aforementioned modules with reference prices.

Table 3.2. Ethernet module parameters comparison

Module	Transmission Standard	Protocols	Interface	Interface Interface	Price
ENC28J60	10Base-T	MAC	SPI	20 MHz	32.70 PLN
W5500	10/100Base-T	MAC, TCP, UDP, ICMP, IPv4, ARP	SPI IGMP, PPPoE	80 MHz	115.00 PLN

Two of the most popular wireless connectivity modules are ESP8266 and ESP32, presented by figures 3.14 and 3.15 respectively. The later module is a newer version of the former one. Both of the modules operate in 2.4 GHz band, with the IEEE 802.11 b/g/n standards. They differ in available speeds, security features and serial interfaces providing connection with the microcontroller. Both modules use pad and THT connectors, allowing the user to solder in the goldpin pins. Both ESP8266 and ESP32 can operate in the softAP (*Soft Access Point*) mode and as a transceiving station (*STA*).

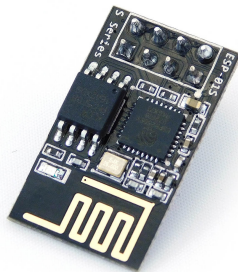


Figure 3.14. ESP8266 module - Nettigo



Figure 3.15. ESP32 module - Botland

ESP8266 provides the communication speed up to 72.2 Mbps. *Firmware* delivered by the module manufacturer implements IPv4, TCP, UDP, HTTP, ICMP and MAC protocols and offers encryption via WEP, WPA/WPA2, TKIP and AES algorithms [17]. In order to communicate with microcontroller, it uses the UART (*Universal Asynchronous Receiver Transmitter*) interface, and uses AT commands in order to control the module. Its operating voltage is around 3.3 V, and two antenna types are available - a microband integrated antenna, or an u.FL port.

ESP32 is a newer version or said ESP8266. The circuit contains a dual core processing unit, which allowed for increasing the transmission speed up to 150 Mbps [18]. Similarly to ESP8266, the manufacturer provides *firmware* which implements IPv4, TCP, UDP, HTTP, ICMP and MAC protocols, but has more extensive security features, extending the encryption algorithm range by PSK/Enterprise, SHA-2, ECC and RSA-4096 algorithms. Some of the variants of the module have integrated

support for IPv6 protocol. There is a possibility for the user to create their custom version of said firmware, containing protocols suited for their needs, using the SDK (*Software Development Kit*) provided by the manufacturer. The module operates with the voltage of 3.3 V. In order to communicate with the microcontroller, user can use either SPI, I²C or UART interfaces. Table 3.3 shows the comparison of the parameters of the modules along with their reference prices.

Table 3.3. Overview of wireless modules parameters

Module	Transmission Standard	Protocol	Interface	Interface Speed	Price
ESP8266	802.11 b/g/n	MAC, TCP, UDP, HTTP, IPv4, ICMP	UART	72,2 Mbps	14.90 PLN
ESP32	802.11 b/g/n	MAC, TCP, UDP, HTTP, IPv4, (IPv6), ICMP	SPI, I ² C, UART	150 Mbps	127.50 PLN

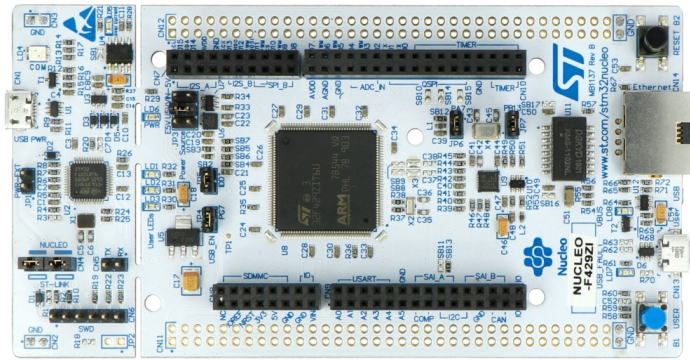


Figure 3.16. Nucleo-F429ZI platform with integrated Ethernet port - Botland

Some of the manufacturers of hardware platform decide to partially implement the TCP/IP stack for their platforms in cases, when they are equipped with integrated network interface (such as 8P8C port). Nucleo-F4 are a good example of such platforms (eg. Nucleo-F429ZI board shown by figure 3.16) equipped with integrated 8P8C port. The HAL (*Hardware Abstract Layer*) software distributed by the ST Microelectronics for said microcontrollers provides the implementation of the network access layer, however it does not offer the higher layers, requiring the user to utilize middleware, such as eg. open source LwIP (*Lightweight IP*) or uIP (*micro IP*) stacks. In order to use the said protocol stacks, they need to be ported first for given microcontroller, or the user needs to use an already available solution if it has been created so far.

The LwIP stack contains the implementation of protocols of application, transport and network layer. Those protocols are as follows [19]:

- 1) IP;

- 2) ICMP;
- 3) UDP;
- 4) DHCP;
- 5) TCP;
- 6) ARP;
- 7) BSD (optionally)

Since version v.2.0.0 the LwIP stack supports both IPv4 and IPv6 addressing.

uIP is an open-source implementation of the TCP/IP stack dedicated for micro-controllers with very limited Flash and RAM memory. According to the document [20] the uIP stack can minimize the RAM usage to a couple hundred bytes. It implements the following protocols:

- 1) TCP;
- 2) UDP;
- 3) IP;
- 4) ICMP;
- 5) ARP

Due to very harsh requirements regarding memory optimization, all application layer protocols are left for implementation by the user. This decision was also made in part due to being unable to predict what specific applications the stack will be used for.

Chapter 4

Practical implementation of the network interface

4.1. Project goals

Following project goals were defined while constructing the practical solution:

- 1) The implementation will consist of a wired network interface using Ethernet and wireless network interface using ESP8266 module;
- 2) The IoT module will receive the network information such as its assigned IP address, subnetwork mask and gateway IP via DHCP service;
- 3) Solution will allow to establish communication with the device using the *ping* command;
- 4) Solution will offer an echo server which will relay back messages received by the IoT module via TCP protocol back to the sender.

4.2. Chosen components

4.2.1. Hardware platform

Due to the already available physical implementation for 8P8C socket and wide availability of reference materials such as documentation and resources produced by the community, Nucleo-F429ZI by ST Microelectronics was chosen as the target hardware platform. In order to implement the wireless interface, the aforementioned ESP8266 module was used due to its small cost.

4.2.2. Environment and programming language

During the analysis of available resources a decision was undertaken to pick STM32-CubeIDE which is the dedicated environment for the chosen hardware platform, as the IDE of choice. This is a result of the availability of embedded configuration tools, which allow for configuring selected parameters of the hardware platform without the

need to manually manipulate the registers of the microcontroller. The programming language was determined to be C.

4.2.3. Verification methods of the system functionality

Verification of the system functionality will happen in a two-step process. First step includes transferring information about the IoT module, such as its IP address, sub-network mask and gateway address via a virtual serial port. Second step consists of an attempt to communicate with the module via *ping* command and communication with the implemented echo server using Hercules SETUP Utility software, which allows communication using the TCP protocol.

4.3. Hardware implementation

4.3.1. Ethernet interface

Thanks to the selected hardware platform, the hardware implementation was already done in-factory on the Nucleo-F429ZI board. The board contains soldered in 8P8C connector with designated RMII interface, as presented on figure 4.1.

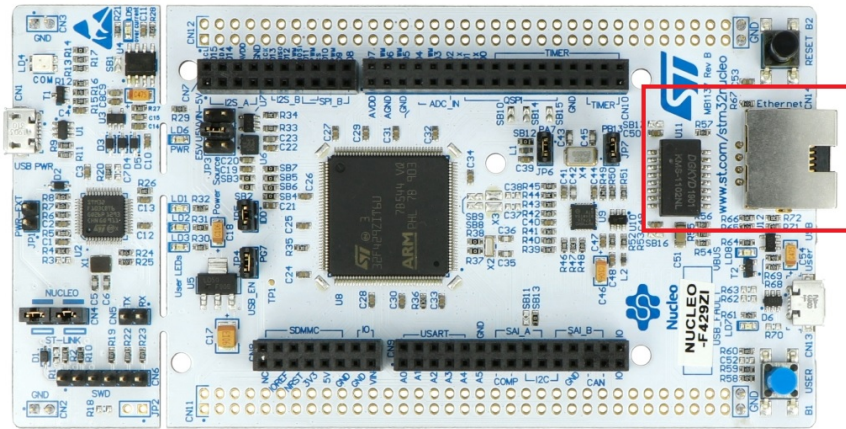


Figure 4.1. 8P8C connector present on the Nucleo-F429ZI board

4.3.2. Wireless interface

In order to communicate with the ESP8266 module, an USART6 serial interface of the STM32-F429ZI microcontroller was used. It is present at pins PC6 and PC7. Additionally a PB3 pin was used in order to provide logical high level to the *enable* input pin of the ESP8266. Figure 4.2 shows the schematic of the connections.

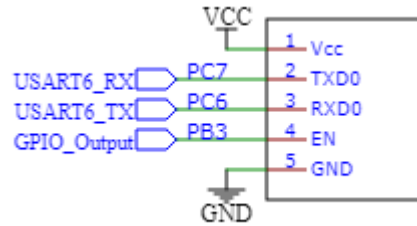


Figure 4.2. ESP8266 connection schematic

4.4. Software implementation

4.4.1. Ethernet interface

The first step of the implementation consisted of initializing the Ethernet interface allowing for correct transmission of the *ping* packet and to signalize the state of the connection via an LED.

Initialization process started by launching the STM32CubeIDE environment and creating a new project selecting STM32L429ZI microcontroller. Figure 4.3 shows the microcontroller platform selection, and figure 4.4 options of the created project.

The screenshot shows the STM32CubeIDE MCU/MPU Selector interface. The selected microcontroller is the STM32F429ZI. The interface displays the following information:

- Part Number:** STM32F429ZI
- Core:** Cortex-M4
- Series:** STM32F4
- Line:** 429
- Package:** LQFP144
- Other:** Active
- Peripheral:** FMC with SDRAM, TFT

The interface also provides a detailed description of the microcontroller's features and specifications, including its performance, memory, and various peripherals.

Part No.	Reference	Marketing	Unit Price f...	Board	Package	Flash	RAM	IO	Freq.
STM32F429	Active	6.48	NUCLEO_STM32	LQFP144	2048 kBytes	256 kBytes	114	180 MHz	
STM32F429ZI	Active	6.48	WLCSP143	2048 kBytes	256 kBytes	114	180 MHz		

Figure 4.3. Microcontroller selection in STM32CubeIDE

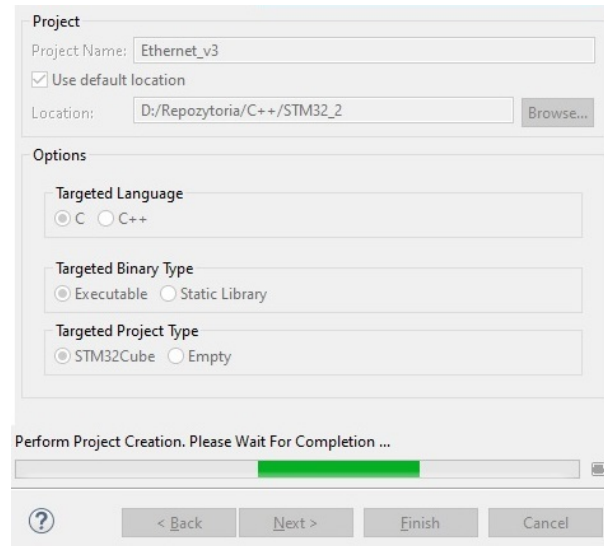


Figure 4.4. Creating the project

The clock signal provided by ST-LINK module being an embedded part of the Nucleo-F429ZI board was used as the system clock. In order to achieve this, High Speed Clock in the RRC tab needed to be set to BYPASS, which was showcased by the figure 4.5. Resulting configuration of the clocking signal was presented on figure 4.6.

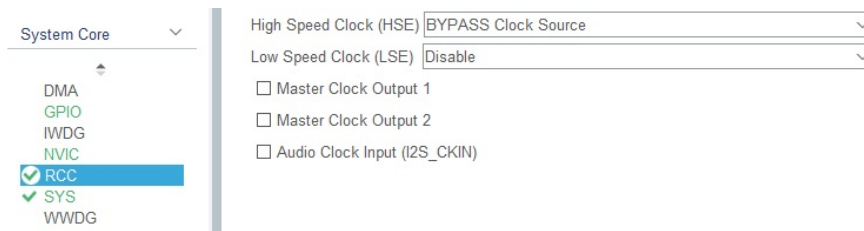


Figure 4.5. Switching the HSE clock into BYPASS mode

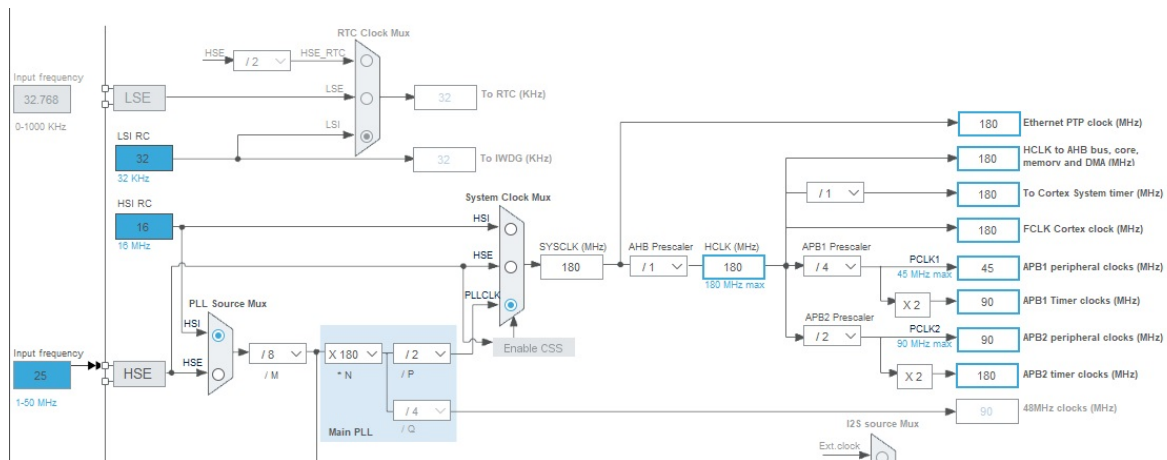


Figure 4.6. Configuring clocking signal after switching HSE to BYPASS mode

The Ethernet interface needs to be initially configured in the *Connectivity* tab, setting its mode to RMII. It is a result of the Nucleo-429ZI board structure, in which the Ethernet connection is realized using RMII interface. The default MAC address of the device is equal to 00:80:E1:00:00:00. During the setup of default values via a configurator, the PHY address of the device was assigned as 1. It needed to be changed to 0. The results were shown on figures 4.7 and 4.8.

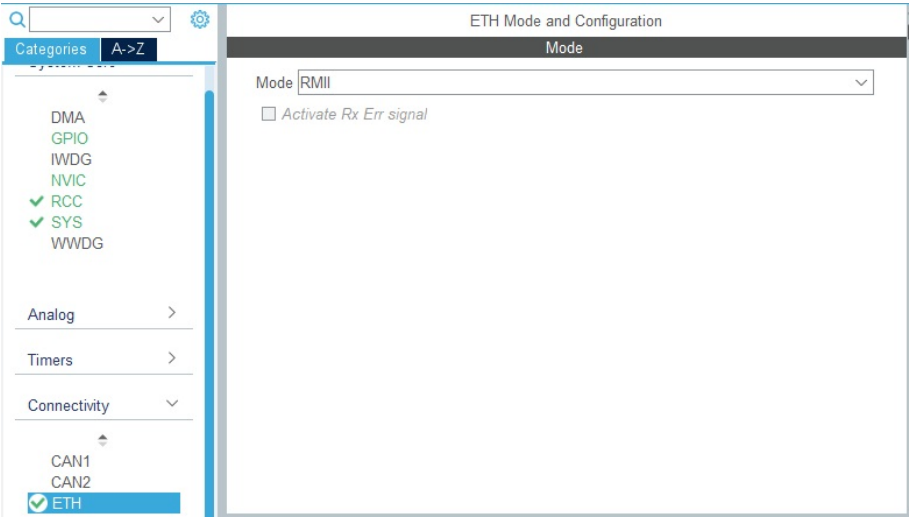


Figure 4.7. Assigning RMII mode to the Ethernet interface

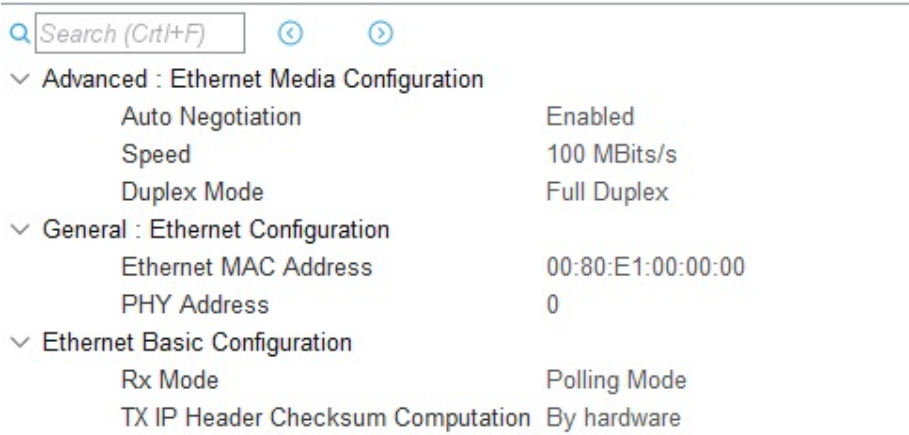


Figure 4.8. Target values of RMII parameters

Due to the connections present on the Nucleo-F429ZI board in accordance to the table presented in the manufacturer documentation [21] the output pins of the microcontroller were assigned in a way showcased in table 4.1. The RMII interface connection schematics are presented by figure 4.9.

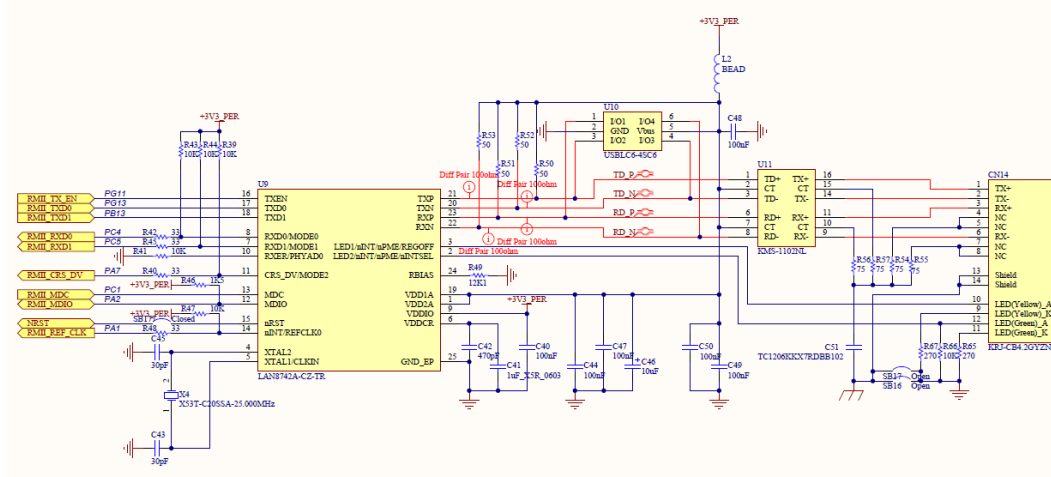


Figure 4.9. RMIi interface connection schematics

Table 4.1. Output line assignment

Pin name	Functionality
PA1	ETH_REF_CLK
PA2	ETH_MDIO
PA7	ETH_CRS_DV
PC4	ETH_RXD0
PC5	ETH_RXD1
PG11	ETH_TX_EN
PG13	ETH_TXD0
PB13	ETH_TXD1
PB14	Red LED
PB0	Green LED
PB7	Blue LED
Pin name	Functionality
PD9	RX line of virtual serial port USART3
PD8	TX line of virtual serial port USART3

The next step included configuring the LwIP stack used for operating the TCP/IP stack protocols. In order to do that, user needs to proceed to the *Middleware* tab as shown on figure 4.10, where the user gains access to various software for selected microcontroller, such as the LwIP stack or freeRTOS real time operating system. The LwIP stack is characterized by big selection of configuration options. In the implementation for the needs of this dissertation, majority of the configuration can be left as default. User needs to make sure however that the LWIP_NETIF_LINK_CALL-BACK, which designates a function called when the Ethernet connector is plugged in to 8P8C socket, is enabled, as shown on figure 4.11.

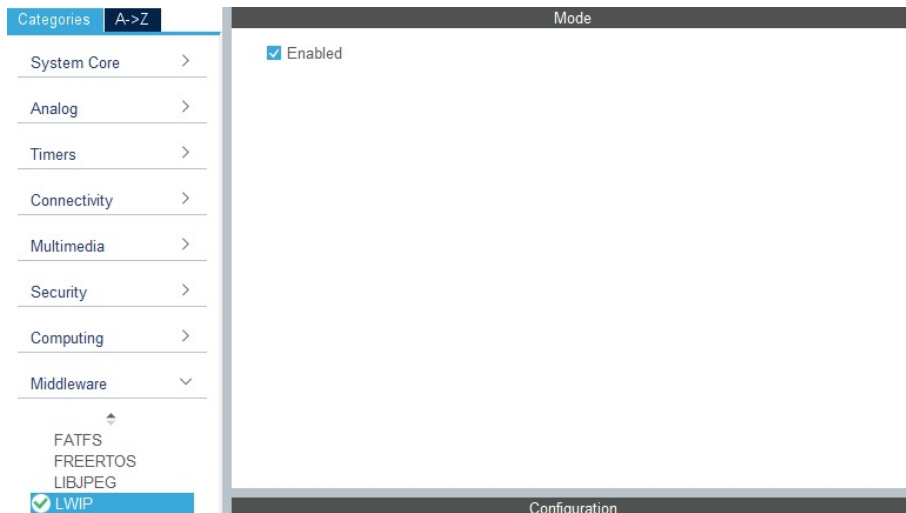


Figure 4.10. Enabling LwIP stack

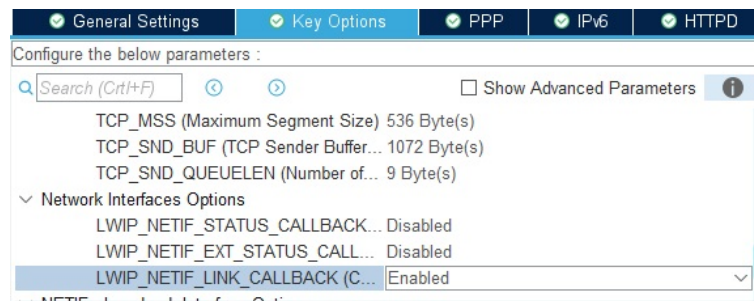


Figure 4.11. LWIP_NETIF_LINK_CALLBACK option

Using the *Clock Configuration* tab, the Ethernet clock signal was set to the frequency of 180 MHz, via the PLL loop and correct prescalers, as presented by figure 4.12.

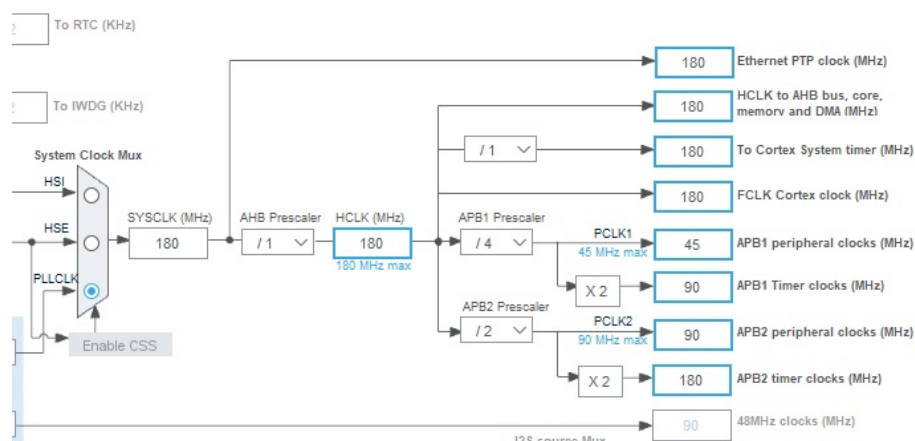


Figure 4.12. Clock configuration

The follow up included generating the code and its modification. During the implementation of the echo server using the LwIP stack and modification of the *MX_LWIP_Process()* function, it was decided to use a sample implementation by a korean Naver blog user nicknamed eziya76 [22].

Listing 4.1. Ethernet implementation *main()* function

```

1  /* Connection structure */
2  extern struct netif gnetif;
3  extern UART_HandleTypeDef huart3;
4
5  /* Char buffer for connection data */
6  char conn_msg_str[24]; // message
7  char ip_str[24];       // IP address
8  char mask_str[24];     // mask
9  char gw_str[24];       // gateway
10 char mac_str[30];      // MAC address
11
12 int main(void)
13 {
14     /* USER CODE BEGIN 1 */
15
16     /* USER CODE END 1 */
17
18     /* MCU Configuration-----*/
19
20     /* Reset of all peripherals, Initializes the Flash interface and the
21      * SysTick. */
22     HAL_Init();
23
24     /* USER CODE BEGIN Init */
25
26     /* USER CODE END Init */
27
28     /* Configure the system clock */
29     SystemClock_Config();
30
31     /* USER CODE BEGIN SysInit */
32
33     /* USER CODE END SysInit */
34
35     /* Initialize all configured peripherals */
36     MX_GPIO_Init();
37     MX_USART3_UART_Init();
38     MX_LWIP_Init();
39     /* USER CODE BEGIN 2 */
40     /* Sending MAC via serial port */
41     send_MAC_address();
42     /* Check connection status */
43     notify_dhcp_status(&gnetif);
44     /* Initialize echo server */
45     init_echo();
46     /* USER CODE END 2 */
47
48     /* Infinite loop */
49     /* USER CODE BEGIN WHILE */
50     while (1)
51     {
52         /* Process data via LwIP stack */
53         MX_LWIP_Process();
54         /* Check connection status */
55         notify_dhcp_status(&gnetif);
56         /* USER CODE END WHILE */
57

```

```

58     /* USER CODE BEGIN 3 */
59 }
60 /* USER CODE END 3 */
61 }

```

Listing 4.1 presents the contents of the *main()* function for the project implementing the Ethernet interface. Part of this function was generated via the code generator based on the STM32CubeIDE environment configuration.

In order to store and format information regarding the connection, such as the IP address, subnetmask, gateway address, MAC address and connection status, 5 globally available buffers were created. This information is transmitted to the overseeing computer via USART3 serial port inside the *notify_dhcp_status()* function.

The *MX_LWIP_Process()* function takes care of receiving the dynamic IP address from the DHCP server enabled in the router and was presented on listing 4.2. The echo server is initialized in the *init_echo()*, repeating a copy of received message to the sender. Listing 4.3 showcases implementation of the *notify_dhcp_status()* function. Listings 4.4 - 4.7 picture the implementation of functions that process information about connection and MAC address into a character string.

Listing 4.2. *MX_LWIP_Process()* function

```

1 void MX_LWIP_Process(void)
2 {
3     /* USER CODE BEGIN 4_1 */
4     /* USER CODE END 4_1 */
5     ethernetif_input(&gnetif);
6
7     /* USER CODE BEGIN 4_2 */
8     /* USER CODE END 4_2 */
9     /* Handle timeouts */
10    sys_check_timeouts();
11
12    /* USER CODE BEGIN 4_3 */
13    ethernetif_set_link(&gnetif);
14    /* If network cable is connected and coonection not set */
15    if(netif_is_link_up(&gnetif) && ! netif_is_up(&gnetif))
16    {
17        /* Establish connection */
18        netif_set_up(&gnetif);
19        /* Get IP from DHCP */
20        dhcp_start(&gnetif);
21    }
22    /* USER CODE END 4_3 */
23 }

```

Listing 4.3. Connection state overseeing function

```

1  /* DHCP address querrying callback */
2  void notify_dhcp_status( struct netif *netif)
3  {
4      /* Connection status flag */
5      static uint8_t connected = 0;
6
7      /* If network cable is connected and IP is different from 0x0 */
8      if (netif_is_link_up(netif) && netif->ip_addr.addr != 0)
9      {
10         /* If previous state was disconnected */
11         if (!connected)
12         {
13             /* Prepare connected message */
14             sprintf(conn_msg_str, "CONNECTED\n\r");
15             /* Prepare IP address message */
16             parse_ip(netif->ip_addr.addr);
17             /* Prepare subnet mask message */
18             parse_mask(netif->netmask.addr);
19             /* Prepare gateway message */
20             parse_gateway(netif->gw.addr);
21             /* Set connection flag */
22             connected = 1;
23             /* Transmit all the data via serial port */
24             HAL_UART_Transmit(&huart3, (unsigned char*)conn_msg_str,
25                             sizeof(conn_msg_str), 10);
26             HAL_UART_Transmit(&huart3, (unsigned char*)ip_str,
27                             sizeof(ip_str), 10);
28             HAL_UART_Transmit(&huart3, (unsigned char*)mask_str,
29                             sizeof(mask_str), 10);
30             HAL_UART_Transmit(&huart3, (unsigned char*)gw_str,
31                             sizeof(gw_str), 10);
32         }
33         /* Turn the red LED off and set green LED on to signalize
34            the connection has been established */
35         HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_SET);
36         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
37     }
38     /* In case the network cable is disconnected or DHCP does not provide
39        an IP address */
40     else
41     {
42         /* If previous state was connected */
43         if (connected)
44         {
45             /* Prepare disconnected message */
46             sprintf(conn_msg_str, "DISCONNECTED\n\r");
47             /* Reset IP address */
48             netif->ip_addr.addr = 0;
49             /* Reset connection flag */
50             connected = 0;
51             /* Transmit disconnection message via serial port */
52             HAL_UART_Transmit(&huart3, (unsigned char*)conn_msg_str,
53                             sizeof(conn_msg_str), 10);
54         }
55         /* Turn off green LED and turn on red LED to signalize
56            disconnection */
57         HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_RESET);

```

```

58     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
59 }
60 }

```

Listing 4.4. Function processing the IP address

```

1  /* PArsing IP address function */
2  void parse_ip(uint32_t ip)
3  {
4      /* IP address parts buffer */
5      uint16_t ip_part[4];
6      /* Extracting IP parts using bitwise operations */
7      for(uint8_t i = 0; i < 4; i++)
8      {
9          ip_part[3 - i] = (ip >> (i * 8)) & 0xFF;
10     }
11     /* Formatting IP address to string */
12     sprintf(ip_str, "IP:_%hu.%hu.%hu.%hu\n\r", ip_part[3],
13             ip_part[2], ip_part[1], ip_part[0]);
14 }

```

Listing 4.5. Function processing the subnetwork mask

```

1  /* Subnet mask parsing function */
2  void parse_mask(uint32_t msk)
3  {
4      /* Buffer for subnet parts */
5      uint16_t msk_part[4];
6      /* Extracting subnet mask parts using bitwise operations */
7      for(uint8_t i = 0; i < 4; i++)
8      {
9          msk_part[3 - i] = (msk >> (i * 8)) & 0xFF;
10     }
11     /* Formatting subnet mask to string */
12     sprintf(mask_str, "Msk:_%hu.%hu.%hu.%hu\n\r", msk_part[3],
13             msk_part[2], msk_part[1], msk_part[0]);
14 }

```

Listing 4.6. Function processing the gateway address

```

1  /* Gateway address parsing function */
2  void parse_gateway(uint32_t gw)
3  {
4      /* Gateway address parts buffer */
5      uint16_t gw_part[4];
6      /* Extracting gateway parts using bitwise operations */
7      for(uint8_t i = 0; i < 4; i++)
8      {
9          gw_part[3 - i] = (gw >> (i * 8)) & 0xFF;
10     }
11     /* Formatting gateway to string format */
12     sprintf(gw_str, "Gw:_%hu.%hu.%hu.%hu\n\r", gw_part[3],
13             gw_part[2], gw_part[1], gw_part[0]);
14 }

```

Listing 4.7. Function transferring the MAC address during the start of the device

```

1  /* Function transferring MAC address */
2  void send_MAC_address(void)
3  {
4      /* Offset for hexadecimal characters */
5      uint8_t hex_offset = 'A' - 10;
6      /* Offset for decimal characters */
7      uint8_t dec_offset = '0';
8      /* MAC address char buffer */
9      char mac_part[12];
10
11     /* Iterating through MAC */
12     for(uint8_t i = 0; i < 6; i++)
13     {
14         /* Extracting MAC address chars */
15         mac_part[i*2] = (gnetif.hwaddr[i] & 0xF0) >> 4;
16         mac_part[i*2+1] = gnetif.hwaddr[i] & 0x0F;
17
18         /* Formatting MAC values to character representation */
19         if(mac_part[i*2] > 10) mac_part[i*2] += hex_offset;
20         else mac_part[i*2] += dec_offset;
21
22         if(mac_part[i*2+1] > 10) mac_part[i*2+1] += hex_offset;
23         else mac_part[i*2+1] += dec_offset;
24     }
25     /* Formatting MAC into a message */
26     sprintf(mac_str, "MAC:_%c%c:%c%c:%c%c:%c%c:%c%c%c\n\n\r",
27             mac_part[0], mac_part[1],
28             mac_part[2], mac_part[3],
29             mac_part[4], mac_part[5],
30             mac_part[6], mac_part[7],
31             mac_part[8], mac_part[9],
32             mac_part[10], mac_part[11]);
33     /* Sending MAC through the serial port */
34     HAL_UART_Transmit(&huart3, (unsigned char*)mac_str, sizeof(mac_str), 10);
35 }

```

In order to control the echo server, a *echo_info* structure was created, which stores information regarding the connection status, retransmission count, pointer to the pcb structure of the LwIP stack and a pointer to the transceiving buffer. Additionally, an enumerating type was created which allows to use connection state names instead of numerical values. Aforementioned elements were placed in a header file listed on listing 4.8.

Listing 4.8. Echo server header file

```

1  /*
2   * echo.h
3   *
4   * Created on: Oct 17, 2021
5   * Author: Kacper Wojciechowski
6   */
7
8  #ifndef INC_ECHO_H_
9  #define INC_ECHO_H_
10
11  #include "lwip/debug.h"

```

```

12 #include "lwip/stats.h"
13 #include "lwip/tcp.h"
14
15 /* Echo server states */
16 enum tcp_state_enum {
17     TCP_STATE_NONE = 0,
18     TCP_STATE_ACCEPTED,
19     TCP_STATE_RECEIVED,
20     TCP_STATE_CLOSING
21 };
22
23 /* Echo server info */
24 struct echo_info {
25     uint8_t state;          // state
26     uint8_t retries;        // retransmission counter
27     struct tcp_pcb* pcb;    // PCB pointer
28     struct pbuf* p;         // packet buffer
29 };
30
31 err_t init_echo(void);
32
33
34 #endif /* INC_ECHO_H */

```

The source file associated with the above header file contains prototypes and implementations of the callback functions, send function and connection shutdown function. They were presented on listing 4.9.

Listing 4.9. A fragment of the echo server source file

```

1  /*
2   * echo.c
3   *
4   *   Created on: Oct 17, 2021
5   *   Author: Kacper Wojciechowski
6   */
7
8  #include "echo.h"
9
10 static struct tcp_pcb* pcb_server; // echoserver pcb
11
12 /* LwIP stack callbacks */
13 static err_t app_callback_accepted(void* arg, struct tcp_pcb* pcb_new,
14                                     err_t err);
15 static err_t app_callback_received(void* arg, struct tcp_pcb* tpcb,
16                                     struct pbuf* p, err_t err);
17 static void app_callback_error(void* arg, err_t err);
18 static err_t app_callback_sent(void* arg, struct tcp_pcb* tpcb,
19                                uint16_t len);
20 static err_t app_callback_poll(void* arg, struct tcp_pcb* tpcb);
21
22 /* Utility function */
23 static void app_send_data(struct tcp_pcb* tpcb, struct echo_info* info);
24 static void app_close_connection(struct tcp_pcb* tpcb,
25                                 struct echo_info* info);

```

Starting the echo server begins with its initialization via the *init_echo()* function, called in the *main()* function before the main loop. It creates a new TCP server

object, connects it to a selected port and registers a connection accept callback. Port 7 was selected for the communication purposes. Listing 4.10. contains the code of the *init_echo()* function.

Listing 4.10. TCP echo server initialization function

```

1  /* Echo server initialization function */
2  err_t init_echo()
3  {
4      err_t err;
5      pcb_server = tcp_new(); // allocating pcb memory
6
7      if(pcb_server == NULL)
8      {
9          /* In case of lack of memory, free allocated memory and
10             return an error */
11          memp_free(MEMP_TCP_PCB, pcb_server);
12          return ERR_MEM;
13      }
14
15      /* Bind server to port 7 */
16      err = tcp_bind(pcb_server, IP_ADDR_ANY, 7);
17      if (err != ERR_OK)
18      {
19          /* In case binding fails, free memory and return
20             an error */
21          memp_free(MEMP_TCP_PCB, pcb_server);
22          return err;
23      }
24
25      /* Listening */
26      pcb_server = tcp_listen(pcb_server);
27      /* Registering connection accept callback */
28      tcp_accept(pcb_server, app_callback_accepted);
29
30      return ERR_OK;
31  }
```

The *app_callback_accepted()* function is called when the connection is accepted by the server. It sets the priority of the created pcb, allocates the *echo_info* structure, sets its initial parameters and registers callbacks for data reception, error and polling. The code of those functions was placed on listing 4.11.

Listing 4.11. Connection accept callback

```

1  /* Connection accept callback */
2  static err_t app_callback_accepted(void* arg, struct tcp_pcb* pcb_new,
3                                     err_t err)
4  {
5      struct echo_info* info;
6
7      /* removing unused parameters warning */
8      (void)arg;
9      (void)err;
10
11     /* Setting new pcb priority */
12     tcp_setprio(pcb_new, TCP_PRIO_NORMAL);
```

```

13
14  /* Allocating echo_info */
15  info = (struct echo_info*)mem_malloc(sizeof(struct echo_info));
16
17
18  if (info == NULL)
19  {
20      /* In case of memory error, close connection and propagate
21       * error */
22      app_close_connection(pcb_new, info);
23      return ERR_MEM;
24  }
25
26  /* Setting connection state to accepted */
27  info->state = TCP_STATE_ACCEPTED;
28  /* Saving pcb pointer */
29  info->pcb = pcb_new;
30  /* Reset retransmission counter */
31  info->retries = 0;
32  /* Reset buffer pointer */
33  info->p = NULL;
34
35  /* Passing pcb_new as an argument */
36  tcp_arg(pcb_new, info);
37  /* Registering data receive callback */
38  tcp_recv(pcb_new, app_callback_received);
39  /* Registering error callback */
40  tcp_err(pcb_new, app_callback_error);
41  /* Registering polling callback */
42  tcp_poll(pcb_new, app_callback_poll, 0);
43
44  return ERR_OK;
45 }

```

The next callback is *app_callback_received()*, called during the data reception from the connected device. It takes care of supervising the buffers, engages sending data back to the sender in the form of echo, and accordingly modifies the connection state saved in the *echo_info* structure. Additionally it registers the *app_callback_sent()* callback. Its code is presented by listing 4.12.

Listing 4.12. Data reception callback function

```

1  /* Data receive callback */
2  static err_t app_callback_received(void* arg, struct tcp_pcb* tpcb,
3                                     struct pbuf* p, err_t err)
4  {
5      /* echo_info pointer */
6      struct echo_info* info;
7      /* error data variable */
8      err_t ret_err;
9
10     /* Asserting echo_info is not null */
11     LWIP_ASSERT("arg!=_NULL", arg != NULL);
12     /* Downcasting the parameter pointer to echo_info pointer */
13     info = (struct echo_info*) arg;
14
15     /* If there is no data to be sent */

```



```
16  if(p == NULL)
17  {
18      /* Change connection state to closing */
19      info->state = TCP_STATE_CLOSING;
20      /* If data buffer is null */
21      if(info->p == NULL)
22      {
23          /* Close connection */
24          app_close_connection(tpcb, info);
25      }
26      /* If there is data to be sent */
27      else
28      {
29          /* Registering data send callback */
30          tcp_sent(tpcb, app_callback_sent);
31          /* Transferring data */
32          app_send_data(tpcb, info);
33      }
34      ret_err = ERR_OK;
35  }
36  /* If there was a reception error */
37  else if(err != ERR_OK)
38  {
39      /* If buffer is present */
40      if(p != NULL)
41      {
42          /* Clearing reception buffer */
43          info->p = NULL;
44          /* Freeing the buffer */
45          pbuf_free(p);
46      }
47      /* Saving error info */
48      ret_err = err;
49  }
50  /* Reception of first batch of data */
51  else if (info->state == TCP_STATE_ACCEPTED)
52  {
53      /* Changing connection status to received */
54      info->state = TCP_STATE_RECEIVED;
55      /* Saving buffer into the echo_info structure */
56      info->p = p;
57      /* Registering data send callback */
58      tcp_sent(tpcb, app_callback_sent);
59      /* Transmitting data */
60      app_send_data(tpcb, info);
61      ret_err = ERR_OK;
62  }
63  /* Reception of subsequent data */
64  else if (info->state == TCP_STATE_RECEIVED)
65  {
66      /* If the buffer is absent */
67      if(info->p == NULL)
68      {
69          /* Saving buffer into the structure */
70          info->p = p;
71          /* Data transmission */
72          app_send_data(tpcb, info);
73      }
```

```

74     /* If there is data to be send, but structure
75        buffer is not empty */
76     else
77     {
78         /* Creating pointer to the buffer */
79         struct pbuf* ptr = info->p;
80         /* Appending the buffer */
81         pbuf_chain(ptr, p);
82     }
83     ret_err = ERR_OK;
84 }
85 /* Data reception when connection is being closed */
86 else if(info->state == TCP_STATE_CLOSING)
87 {
88     /* Recommending the window size */
89     tcp_recved(tpcb, p->tot_len);
90     /* Clearing and freeing the buffer */
91     info->p = NULL;
92     pbuf_free(p);
93     ret_err = ERR_OK;
94 }
95 /* Reception done */
96 else
97 {
98     /* Recommending the window size */
99     tcp_recved(tpcb, p->tot_len);
100    /* Clearing and freeing the buffer */
101    info->p = NULL;
102    pbuf_free(p);
103    ret_err = ERR_OK;
104 }
105 return ret_err;
106 }

```

Function *app_callback_sent()* oversees the situation after executing transmission. In case of subsequent data present, it initializes another transmission. *app_callback_poll* function operates in a similar fashion. Listings 4.13 and 4.13 present their code.

Listing 4.13. Data transmission callback

```

1  /* Sending callback */
2  static err_t app_callback_sent(void* arg, struct tcp_pcb* tpcb, uint16_t len)
3  {
4      /* Pointer to the echo_info struct */
5      struct echo_info* info;
6      /* Silencing warnings */
7      (void)len;
8
9      /* Downcasting the echo_info pointer */
10     info = (struct echo_info*) arg;
11     /* Resetting the retransmission counter */
12     info->retries = 0;
13
14     /* If there is data to be sent */
15     if(info->p != NULL)
16     {
17         /* Registering the send callback */
18         tcp_sent(tpcb, app_callback_sent);

```

```

19     /* Data transmission */
20     app_send_data(tpcb, info);
21 }
22 /* If there is no data to be sent */
23 else
24 {
25     /* If the connection is being closed */
26     if(info->state == TCP_STATE_CLOSING)
27     {
28         /* Closing the connection */
29         app_close_connection(tpcb, info);
30     }
31 }
32 return ERR_OK;
33 }

```

Listing 4.14. Pooling callback

```

1  /* Polling callback */
2  static err_t app_callback_poll(void* arg, struct tcp_pcb* tpcb)
3  {
4      /* Pointer to echo_info */
5      struct echo_info* info;
6      /* Downcasting echo_info pointer */
7      info = (struct echo_info*) arg;
8      /* If it did not downcast properly */
9      if(info == NULL)
10     {
11         /* Aborting the connection and returning an error */
12         tcp_abort(tpcb);
13         return ERR_ABRT;
14     }
15     /* If there is data to transmit */
16     if(info->p != NULL)
17     {
18         /* Registering data send callback */
19         tcp_sent(tpcb, app_callback_sent);
20         /* Sending data */
21         app_send_data(tpcb, info);
22     }
23     /* If there is no data to be sent */
24     else
25     {
26         /* If connection is being closed */
27         if(info->state == TCP_STATE_CLOSING)
28         {
29             /* Disconnecting */
30             app_close_connection(tpcb, info);
31         }
32     }
33     return ERR_OK;
34 }

```

The *app_callback_error()* is the last callback, engaged when a transmission error occurs. It is responsible for signaling error using the blue LED and for de-allocating the *echo_info* structure. Its code is shown on listing 4.15.

Listing 4.15. Error callback

```

1  /* Error callback */
2  static void app_callback_error(void* arg, err_t err)
3  {
4      /* echo_info pointer */
5      struct echo_info* info;
6      /* Silence warnings */
7      (void)err;
8
9      /* Downcasting the echo_info pointer from the parameter */
10     info = (struct echo_info*) arg;
11     /* If the echo_info pointer was properly downcasted */
12     if (info != NULL)
13     {
14         /* Release allocated memory */
15         mem_free(info);
16     }
17     /* Turn on the blue LED to signalize error */
18     HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
19 }

```

The key element of the echo server is a function carrying out the transmission of the data back to the sender. This task is placed upon the *app_send_data()* function, utilizing the *tcp_write()* function of the LwIP stack. It allows the user to transmit a series of packets and oversees the correctness of the transmission, carrying out a retransmission if the need arises. Listing 4.16 presents the function content.

Listing 4.16. Data transmission function

```

1  /* Data transmission callback */
2  static void app_send_data(struct tcp_pcb* tpcb, struct echo_info* info)
3  {
4      /* Buffer for data to be sent */
5      struct pbuf* ptr;
6      err_t wr_err = ERR_OK;
7
8      /* As long as no error occurred, there is data to be sent, and
9       data length is less than the send buffer length */
10     while((wr_err == ERR_OK) && (info->p != NULL)
11           && (info->p->len <= tcp_sndbuf(tpcb)))
12     {
13         /* Copying the buffer pointer */
14         ptr = info->p;
15         /* Data transmission */
16         wr_err = tcp_write(tpcb, ptr->payload, ptr->len,
17                           TCP_WRITE_FLAG_COPY);
18
19         /* If data was sent successfully */
20         if(wr_err == ERR_OK)
21         {
22             /* Variable for the buffer length */
23             uint16_t plen;
24             uint8_t freed;
25
26             /* Saving the buffer length */
27             plen = ptr->len;
28             /* Moving buffer to the next packet */

```

```

29     info->p = ptr->next;
30
31     /* If there is a chain buffer */
32     if(info->p != NULL)
33     {
34         /* Increase reference counter */
35         pbuf_ref(info->p);
36     }
37
38     do
39     {
40         /* Free the old buffer */
41         freed = pbuf_free(ptr);
42     }
43     while(freed == 0);
44
45     /* Recommending window size */
46     tcp_recved(tpcb, plen);
47 }
48 /* If transmission failed */
49 else
50 {
51     /* Recovering the buffer pointer */
52     info->p = ptr;
53     /* Increasing the retransmission counter */
54     info->retries++;
55 }
56 }
57 }

```

After the connected device is done operating the echo server, the connection must be closed, which is the task of the *app_close_connection()* function. It de-registers previously set up functions, closes the connection and releases related resources. Its code is shown on listing 4.17.

Listing 4.17. Connection close function

```

1  /* Connection close callback */
2  static void app_close_connection(struct tcp_pcb* tpcb, struct echo_info* info)
3  {
4      /* De-registering callbacks */
5      tcp_arg(tpcb, NULL);
6      tcp_sent(tpcb, NULL);
7      tcp_recv(tpcb, NULL);
8      tcp_err(tpcb, NULL);
9      tcp_poll(tpcb, NULL, 0);
10
11     /* If echo_info is present */
12     if(info != NULL)
13     {
14         /* Freeing the memory */
15         mem_free(info);
16     }
17
18     /* Closing connection */
19     tcp_close(tpcb);
20 }

```

4.4.2. Wi-Fi wireless interface

In order to separate the implementation of Ethernet and wireless interfaces, a new project was created in the STM32CubeIDE.

The implementation of the Wi-Fi wireless interface begins with preparing the STM32-F429ZI microcontroller configuration. The Wi-Fi module used here communicates with the microcontroller using the UART interface, by exchanging the AT standard commands. As of such, the configuring process starts with activating two of the UARD interfaces:

- USART3 - serial port for communicating with the overseeing computer;
- USART6 - serial port for communicating with the Wi-Fi module.

Both of the ports were configured to operate with the baudrate of 115200, 8-bit word length, no parity control and a singular stop bit. The configuration in question was shown on figure 4.13.

▼ Basic Parameters	
Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
▼ Advanced Parameters	
Data Direction	Receive and Transmit
Over Sampling	16 Samples

Figure 4.13. USART3 and USART6 configuration

The next step consists of setting up the HSE in the System Core section of the RRC tab in the BYPASS mode, similarly as in case of the Ethernet interface, in order to use the ST-Link clock signal for clocking the microcontroller.

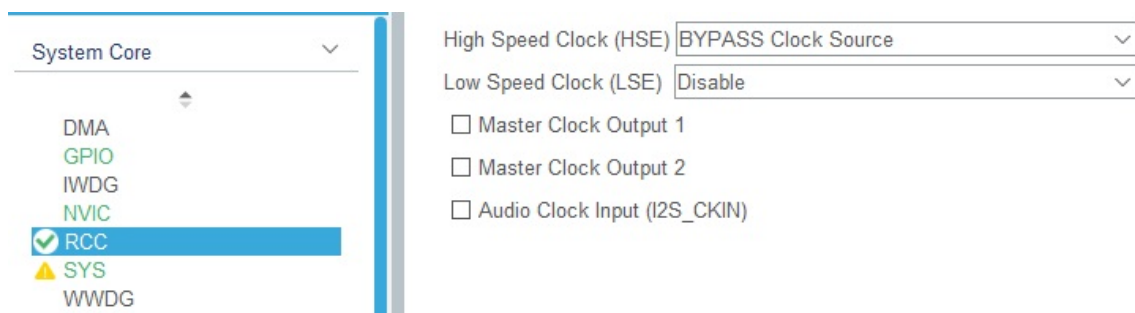


Figure 4.14. HSE clock setup

In the *Clock Configuration* tab, the system clock frequency was set via the configurator to 180 MHz, as shown on figure 4.15.

The last step of the configuration process required configuring one of the GPIO ports in the output mode, in order to control the *enable* line of the Wi-Fi module. In order to activate the module and enable operating it, the line required a high logical state. Line PB3 was used for this purpose, keeping its default parameters,

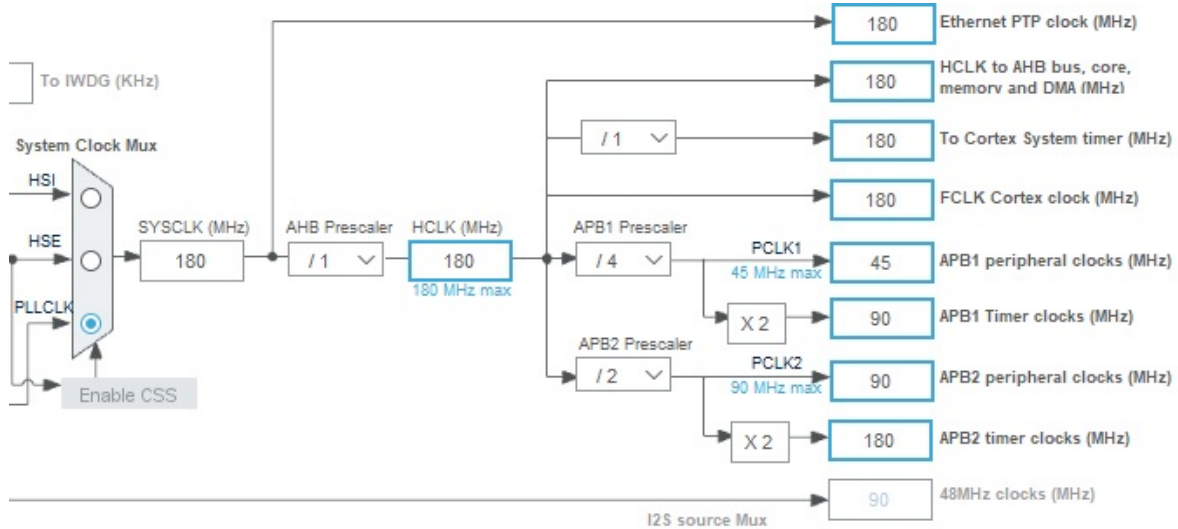


Figure 4.15. Target frequencies in the clock manager

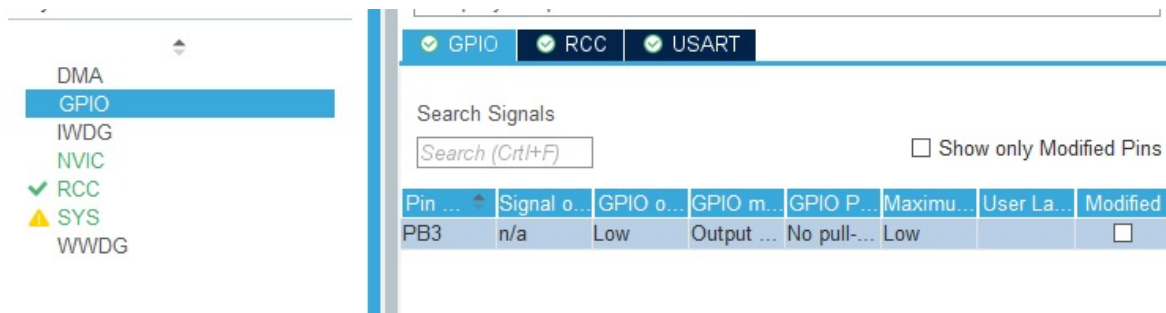


Figure 4.16. Configured GPIO lines

PB3 Configuration :

GPIO output level	Low
GPIO mode	Output Push Pull
GPIO Pull-up/Pull-down	No pull-up and no pull-down
Maximum output speed	Low
User Label	

Figure 4.17. PB3 line parameters

for which the monitoring and modification options are available in the GPIO tab of the *System Core* section. They were presented on figures 4.16 and 4.17.

The ESP8266 module contains a functioning implementation of the TCP/IP stack, and as such enabling the LwIP stack in the platform configurator was not required. Based on this configuration, the source was generated, which was subsequ-

ently extended with snippets that configure the Wi-Fi module and implement the echo server functionality. Listing 4.18 shows the contents of the *main()* function and the globally available elements.

Listing 4.18. *main()* function and global elements

```

1  /* Command count macro */
2  #define COMMANDS 12
3
4  /* Module feedback buffer */
5  char data_r[1000];
6  /* Received data buffer */
7  char message[100];
8
9  /* Echo message ready flag */
10 int ready_to_send = 0;
11
12 /* Configuration commands LUT */
13 char* config[] = {
14     "AT+RST\r\n",           /* Reset module */
15     "ATE0\r\n",             /* Disable command echo */
16     "AT\r\n",               /* Communication test */
17     "AT+GMR\r\n",           /* Firmware version */
18     "AT+CWMODE=1\r\n",       /* Client operation mode */
19     "AT+CIPMODE=0\r\n",      /* Selecting received data
20                               * format */
21     "AT+CIPMUX=1\r\n",       /* Setting multiconnectivity */
22     "AT+CIPSERVER=1,7\r\n",  /* Enable server and
23                               * bind to port 7 */
24     "AT+CWMODE=?\r\n",       /* Check operating mode */
25     "AT+CWJAP=\"marx\", \"*****\r\n", /* Connect to Wi-Fi */
26     "AT+CIPSTA?\r\n",        /* Check Wi-Fi connection
27                               * status */
28     "AT+CIFSR\r\n"          /* Display IP address */
29 };
30
31 int main(void)
32 {
33     /* USER CODE BEGIN 1 */
34
35     /* USER CODE END 1 */
36
37     /* MCU Configuration-----*/
38
39     /* Reset of all peripherals, Initializes the Flash interface and
40      * the SysTick. */
41     HAL_Init();
42
43     /* USER CODE BEGIN Init */
44
45     /* USER CODE END Init */
46
47     /* Configure the system clock */
48     SystemClock_Config();
49
50     /* USER CODE BEGIN SysInit */
51
52     /* USER CODE END SysInit */

```



```

53
54  /* Initialize all configured peripherals */
55  MX_GPIO_Init();
56  MX_USART3_UART_Init();
57  MX_USART6_UART_Init();
58  /* USER CODE BEGIN 2 */
59  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
60
61  /* USER CODE END 2 */
62
63  /* Infinite loop */
64  /* USER CODE BEGIN WHILE */
65
66  /* Iterate thorough the configuration commands LUT configuring the module */
67  for(i = 0; i < COMMANDS; i++)
68  {
69      /* Send configuration command */
70      HAL_UART_Transmit(&huart6, (uint8_t*)(config[i]), strlen(config[i]), 10);
71      /* Clear reception buffer */
72      clr_buffer(data_r, 1000);
73      /* Receive feedback */
74      HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r), 100);
75
76      /* Await for the module wake up after restart */
77      if(i == 0)
78      {
79          HAL_Delay(1000);
80      }
81
82      /* Await IP address reception from DHCP */
83      while(is_busy())
84      {
85          /* Await 1 second */
86          HAL_Delay(1000);
87          /* Query for IP address, subnet mask and gateway */
88          HAL_UART_Transmit(&huart6, (uint8_t*)(config[i]), strlen(config[i]),
89                          10);
90          /* Clear reception buffer */
91          clr_buffer(data_r, 1000);
92          /* Receive feedback */
93          HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r),
94                          100);
95      }
96      /* Transmit received feedback to the overseeing computer
97       using serial port */
98      HAL_UART_Transmit(&huart3, (unsigned char*)data_r, sizeof(data_r),
99                      10);
100     HAL_UART_Transmit(&huart3, (unsigned char*)"\r\n\n", sizeof("\r\n\n"),
101                     10);
102
103 }
104
105 while (1)
106 {
107     /* USER CODE END WHILE */
108     /* Clear reception buffer */
109     clr_buffer(data_r, 1000);
110     /* Read module feedback */

```

```

111     HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r), 100);
112     /* If echo message is ready to be sent */
113     if(ready_to_send)
114     {
115         /* Send echo data */
116         HAL_UART_Transmit(&huart6, (uint8_t*)message, sizeof(message), 10);
117         /* Reset echo message ready flag */
118         ready_to_send = 0;
119     }
120     /* Otherwise, if received message is not empty */
121     else if (data_r[0] != 0)
122     {
123         /* Prepare to send an echo data */
124         prepare_echo();
125         /* Send received feedback through the serial port */
126         HAL_UART_Transmit(&huart3, (unsigned char*)data_r, sizeof(data_r),
127                             10);
128         HAL_UART_Transmit(&huart3, (unsigned char*)"\r\n\n",
129                             sizeof("\r\n\n"), 10);
130     }
131
132     /* USER CODE BEGIN 3 */
133 }
134 /* USER CODE END 3 */
135 }

```

The global elements were defined as:

- character buffer for reception from the Wi-Fi module;
- character buffer for formatted message to be sent back;
- macro defining the Wi-Fi configuration commands amount;
- look up table containing the commands for configuring the Wi-Fi module.

The result of each of the configuration commands was described via comments inside listing 4.18. For privacy protection reasons, the password contained in the listing for the *AT+CWJAP* command was censored with asterisks.

The *main()* function consists of several key sections. The first one is the generated configuration section for each of the interfaces and microcontroller components, such as GPIO, system clock and USART interfaces. This section was extended with setting the PB3 line connected to the *enable* input of the Wi-Fi module into high logical state.

Second section is the configuration loop. Is is tasked with performing the reset and configuration of the Wi-Fi module in a way which enables connection and two-way transmission of messages. These commands are subsequently:

- *AT+RST* - module reset;
- *ATE0* - disabling the configuration commands echo;
- *AT* - communication test with the module;
- *AT+GMR* - acquiring module firmware version;
- *AT+CWMODE=1* - configuring the module into client mode;
- *AT+CIPMODE=0* - configuring the received message format as "+IPD,<channel>,<byte count>,<data>";
- *AT+CIPMUX=1* - enabling multiconnectivity;
- *AT+CIPSERVER=1,7* - configuring the server on port 7;
- *AT+CWMODE=?* - verifying the operating mode;
- *AT+CWJAP=<SSID>,<password>* - connecting to the Wi-Fi network;
- *AT+CIPSTA?* - querying connection status;
- *AT+CIFSR* - acquiring the server IP address;

After sending the module reset command, the microcontroller awaits 1 second before transmitting subsequent commands. Each transmission consists of sending the command to the module, clearing the reception buffer using the *clr_buffer()* function presented on listing 4.19, and receiving the feedback from the module. After the *AT+CWJAP* query, the microcontroller awaits acquiring an IP address from DHCP in 1-second intervals, by querying the connection status via the *is_busy()* function shown on listing 4.20. As long as the status informs that the module is busy, the process is repeated. Feedback from the module is relayed to the overseeing computer via the serial port.

Listing 4.19. Reception buffer clear function

```

1  /* Buffer clear function */
2  static void clr_buffer(char* buff, int size)
3  {
4      int i;
5      /* Overriding entire buffer with default value */
6      for(i = 0; i < size; i++)
7      {
8          buff[i] = 0;
9      }
10 }
```

Listing 4.20. Wi-Fi module state query function

```

1  /* DHCP address polling function */
2  static int is_busy(void)
3  {
4      /* Return variable */
5      int ret = 0;
6      int i;
7      /* Search for "busy" in module feedback */
8      for(i = 0; i < sizeof(data_r); i++)
9      {
10         if(data_r[i] == 'b' && data_r[i+1] == 'u' && data_r[i+2] == 's'
11            && data_r[i+3] == 'y')
12         {
13             /* If the phrase was found, inform that the module is busy
14              * waiting for DHCP */
15             ret = 1;
16             break;
17         }
18     }
19     /* Return module busy status */
20     return ret;
21 }

```

The third section of the *main()* function contains the operating loop, which acts as the echo server functionality. Inside it, a reception buffer is cleared and data is read from the module when an event such as establishing connection or data reception occurs. In case of receiving a non-empty feedback message, the echo functionality is prepared inside the *prepare_echo()* function, which code can be found in listing 4.21.

Listing 4.21. Preparing the echo functionality

```

1  /* Prepare microcontroller for sending echo messages */
2  static void prepare_echo(void)
3  {
4      /* Message length buffer - messages up to 99 chars */
5      char byte_count[3];
6      /* Data send command buffer */
7      char command[20];
8      int i;
9
10     /* Checking whether feedback from the module informs about
11      * receiving a message by the module */
12     /* Starting from index 2 is a result of an offset present at
13      * the message start */
14     if(data_r[2] == '+' && data_r[3] == 'I' && data_r[4] == 'P'
15        && data_r[5] == 'D')
16     {
17         /* Assigning index of the first digit of char count in
18          * received message */
19         i = 9;
20         /* Extracting char count from received message */
21         while(data_r[i] != ':')
22         {
23             byte_count[i-9] = data_r[i];
24             i++;
25         }

```

```

26     /* Closing the string */
27     byte_count[i-9] = '\0';
28     /* Setting the index of first letter of received message */
29     i++;
30     /* Extracting received message */
31     while(data_r[i] != 0)
32     {
33         message[i-11] = data_r[i];
34         i++;
35     }
36     /* Closing the message */
37     message[i++]='\r';
38     message[i++]='\n';
39     message[i] = '\0';
40     /* Formatting the command for sending data */
41     sprintf(command, "AT+CIPSEND=0,%d\r\n", atoi(byte_count));
42     /* Send command to the module */
43     HAL_UART_Transmit(&huart6, (uint8_t*)(command), strlen(command), 10);
44     /* Set ready flag to send an echo message */
45     ready_to_send=1;
46 }
47 }

```

If the received feedback message starts with the "+IPD" sequence, the program extracts the number of bytes and its content in order to properly format it and prepare it for sending back. Readiness for sending is stated by the *ready_to_send* flag. In case of extracting the message, a "send data" command is transmitted, followed by the message content in the third section of the *main()* function. After performing the echo transmission, the *ready_to_send* flag is reset/

4.5. Solution tests

4.5.1. Ethernet interface

During the testing phase, following functionalities were validated:

- communication with the overseeing computer via the serial port, acquiring information about the MAC address and the connection state;
- exchanging the ping via Windows command prompt;
- connection and two-way exchange via Hercules SETUP Utility software.

Figures 4.18, 4.19 and 4.20 showcase the results of carried out tests. The first action included acquiring the IP address of the module via USART serial interface. Subsequently, the connection could be validated using the *ping* command, and the echo server functionality could be verified using the Hercules SETUP Utility. All of the tests were successful, confirming the proper functioning of the Ethernet network interface.

```
HAC: 00:80:E1:00:00:00  
  
CONNECTED  
IP: 192.168.1.179  
Mask: 255.255.255.0  
Gateway: 192.168.1.1  
DISCONNECTED  
CONNECTED  
  
IP: 192.168.1.179  
Mask: 255.255.255.0  
Gateway: 192.168.1.1
```

Figure 4.18. Serial port communication

```
Wiersz poleceń  
Microsoft Windows [Version 10.0.19042.1415]  
(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.  
  
C:\Users\Gerwant>ping 192.168.1.179  
  
Pinging 192.168.1.179 with 32 bytes of data:  
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255  
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255  
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255  
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255  
  
Ping statistics for 192.168.1.179:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
    Approximate round trip times in milli-seconds:  
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Figure 4.19. Verifying connection via ping

4.5.2. Wi-Fi wireless interface

The figure 4.21 showcases a picture of the prepared circuit. Similarly to the Ethernet interface, the same set of tests was performed. Due to the reset of the device, on picture 4.22 some unreadable messages can be seen, caused by the different speed of transmission during the bootloader execution of the module. Just like in the case of the Ethernet interface, first the IP address of the module was acquired via an USART interface, which was shown on figure 4.22 and 4.23. Subsequently communication tests were carried out using the *ping* command, which is shown on figure 4.24 and the echo server functionality using the Hercules SETUP Utility software shown on figure 4.25. All tests were successful.

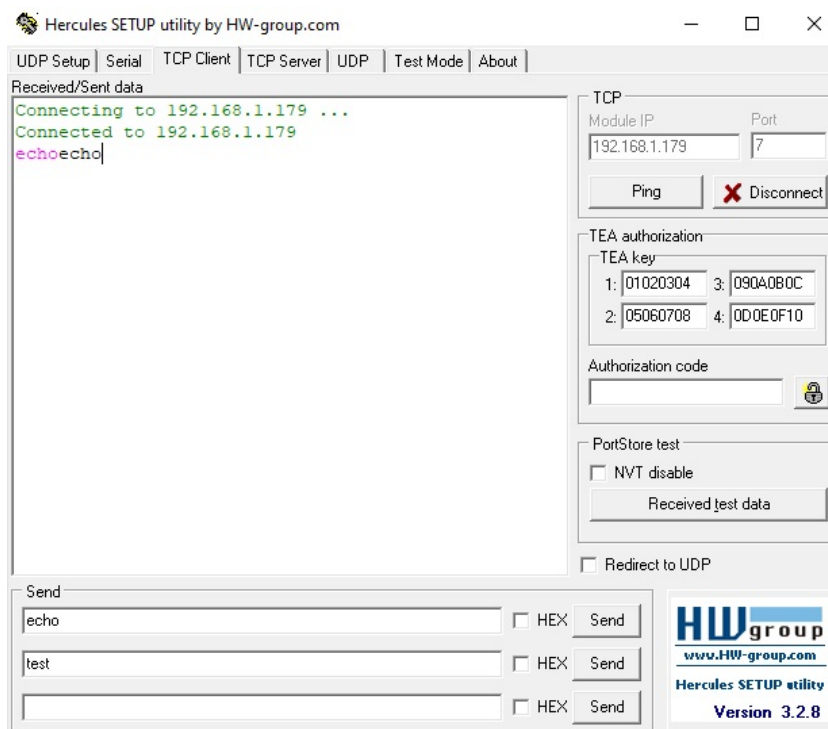


Figure 4.20. Two-way communication validation using Hercules SETUP Utility

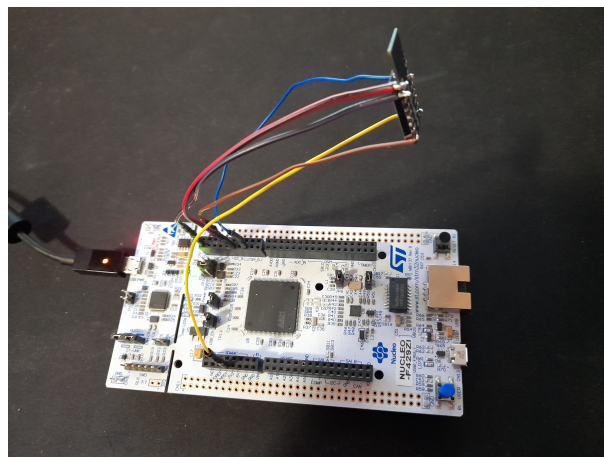


Figure 4.21. Constructed circuit

```

AT+RST
OK
i0f0p0ng0r0x0bb
1
lg0r0i0ft0ip0i0i
1'i0ft0ip0g0

ATE0
OK

OK

AT version:1.2.0.0(Jul 1 2016 20:04:45)
SDK version:1.5.4.1(39cb9a32)
Ai-Thinker Technology Co. Ltd.
Dec 2 2016 14:21:16

OK

OK

OK

```

Figure 4.22. Communication with overseeing computer pt. 1

```

OK

+CHMODE:(1-3)
OK

+CIPSTA:ip:"192.168.1.51"
+CIPSTA:gateway:"192.168.1.1"
+CIPSTA:netmask:"255.255.255.0"
OK

+CIFSR:STAIP,"192.168.1.51"
+CIFSR:STAMAC,"e8:db:84:84:06:cd"
OK

0,CONNECT

+IPD,0,4:echo

busy s...

Recv 4 bytes

SEND OK

0,CLOSED

```

Figure 4.23. Communication with overseeing computer pt. 2

```

C:\Users\Gerwant>ping 192.168.1.51

Pinging 192.168.1.51 with 32 bytes of data:
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128
Reply from 192.168.1.51: bytes=32 time=8ms TTL=128
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.1.51:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 8ms, Average = 2ms

```

Figure 4.24. Testing connectivity using *ping*

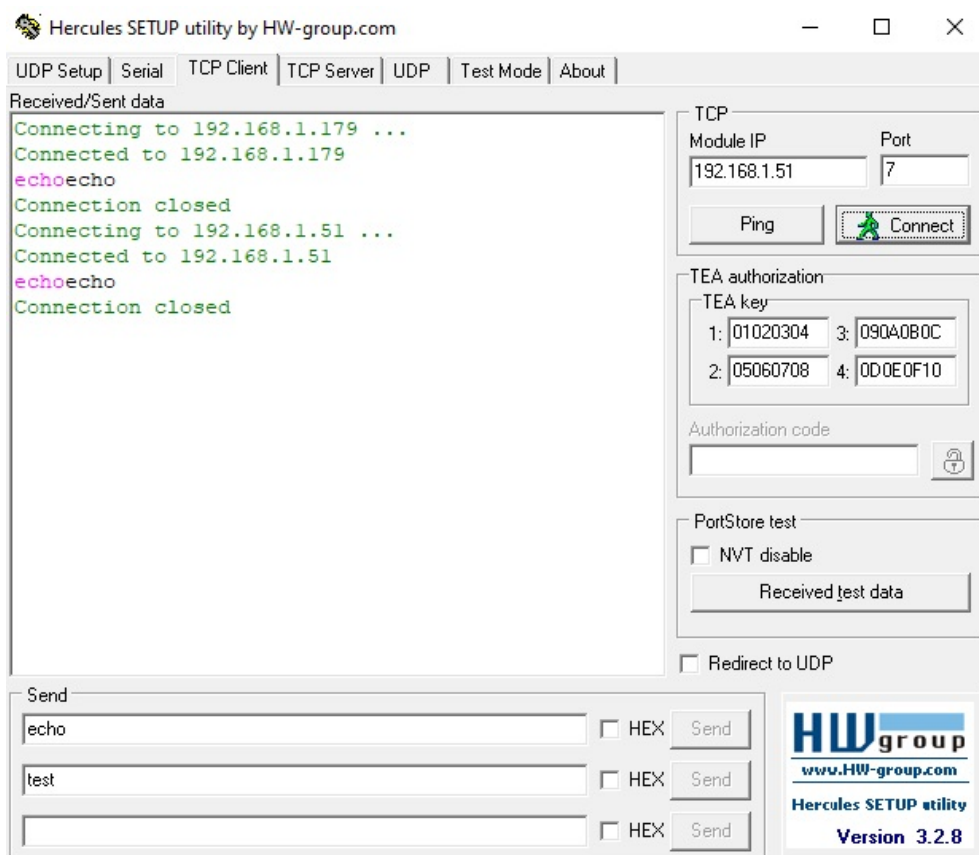


Figure 4.25. Two-sided communication using Hercules SETUP Utility software

Chapter 5

Results discussion and conclusions

As a result of the analysis of existing implementation solutions for network interfaces and the analysis of TCP/IP stack structure, it was decided which hardware platform was chosen for the task of implementing an IoT module as an echo server which serves network connections in the practical aspect of this dissertation.

During the making of this dissertation, following conclusions and observations were drafted:

- There is a wide arrangement of commercially available solutions for implementing network interfaces, both in case of wired and wireless connectivity;
- Some of the available hardware platforms contain ready to use implementations of the physical part of the interface, such as soldered in 8P8C connectors or Wi-Fi modules placed onboard the PCB;
- There are several versions of TCP/IP stack implementation, which differs in such terms as offered functionalities, memory usage and processing time. LwIP and uIP are examples of such open-source TCP/IP stack implementations.
- Despite using the same frequency of the system clock, based on observing the timings of information logs appearing in Hercules SETUP Utility and the serial port terminal, and based on the roundtrip times measured by the *ping* command, the Wi-Fi interface was characterized by increased delay when transferring echo messages, with regards to the Ethernet interface. In case of the echo functionality and communication using serial port, most likely the need to communicate with the Wi-Fi module by the microcontroller using UART interface and its data processing speed are the primary causes.

Bibliography

- [1] *Internet of Things Global Standard Initiative*.
<https://www.itu.int/en/ITU-T/gsi/iot/Pages/default.aspx>, July 2021.
- [2] *Model OSI/ISO*. https://kisi.pcz.pl/Robert_Nowicki/psk-d/slides/030%20Model%20SI.pdf, Aug. 2021.
- [3] *Difference Between TCP/IP and OSI Model*.
<https://techdifferences.com/difference-between-tcp-ip-and-osi-model.html>, Sept. 2021.
- [4] *Protokoły TCP/IP*.
<http://www.crypto-it.net/pl/teoria/protokoly-tcp-ip.html>, Sept. 2021.
- [5] *HTTP*. <https://developer.mozilla.org/pl/docs/Web/HTTP>, Sept. 2021.
- [6] *Protokół SMTP (Simple Mail Transfer Protocol)*.
<https://www.ibm.com/docs/pl/i/7.3?topic=information-smtp>, Sept. 2021.
- [7] *Różnice pomiędzy protokołami POP3 i IMAP*.
https://www.uci.umk.pl/index.php/R%C3%B3%C5%BCnice_pomi%C4%99dzy_protoko%C5%82ami_POP3_i_IMAP, Sept. 2021.
- [8] *TELNET - słownik pojęć technicznych*.
https://www.atel.com.pl/slownik_produk_t_ref.php?haslo=TELNET, Sept. 2021.
- [9] *Protokół SNMP (Simple Network Management Protocol)*. <https://www.ibm.com/docs/pl/spectrum-control/5.3.3?topic=standards-simple-network-management-protocol>, Sept. 2021.
- [10] *IRC*. <http://www.irc.pl/>, Sept. 2021.
- [11] *ARP*. https://www.atel.com.pl/slownik_produk_t_ref.php?haslo=ARP, Sept. 2021.
- [12] *8P8C*. <https://pl.wikipedia.org/wiki/8P8C>, Sept. 2021.
- [13] *Reduced Media-Independent Interface*.
https://en.wikipedia.org/wiki/Media-independent_interface, Aug. 2021.
- [14] Rafał Kozik. “Ethernet w FPGA”. In: *Programista* 91.4 (2020), pp. 18–32.
- [15] *ENC28J60 Data Sheet. Stand-Alone Ethernet Controller with SPI Interface*. Sept. 2021.

- [16] *W5500 Datasheet*. Sept. 2021.
- [17] *ESP8266 - co warto wiedzieć ?* <https://forbot.pl/blog/leksykon/esp8266>, Sept. 2021.
- [18] *ESP32 - co warto wiedzieć ?* <https://forbot.pl/blog/leksykon/esp32>, Sept. 2021.
- [19] *Lightweight TCP/IP (lwIP) Stack*. <https://www.analog.com/en/design-center/evaluation-hardware-and-software/software/adswp-lwip.html>, Sept. 2021.
- [20] *Using the Stellaris Ethernet Controller with Micro IP (uIP)*. <https://www.ti.com/lit/an/spma026b/spma026b.pdf?ts=1631990328536>, Sept. 2021.
- [21] *UM1974 User Manual STM32 Nucleo-144 Boards*. Nov. 2021.
- [22] *[STM32 HAL] Ethernet*. Dec. 2021.